

UNIT V VISUALIZATION WITH MATPLOTLIB

Importing Matplotlib – Simple line plots – Simple scatter plots – visualizing errors – density and contour plots – Histograms – legends – colors – subplots – text and annotation – customization – three dimensional plotting - Geographic Data with Basemap– Visualization with Seaborn.

General Matplotlib Tips

We'll now take an in-depth look at the Matplotlib tool for visualization in Python. Matplotlib is a multiplatform data visualization library built on NumPy arrays, and designed to work with the broader SciPy stack. It was conceived by John Hunter in 2002, originally as a patch to IPython for enabling interactive MATLAB-style plotting via gnuplot from the IPython command line. IPython's creator, Fernando Perez, was at the time scrambling to finish his PhD, and let John know he wouldn't have time to review the patch for several months. John took this as a cue to set out on his own, and the Matplotlib package was born, with version 0.1 released in 2003. It received an early boost when it was adopted as the plotting package of choice of the Space Telescope Science Institute (the folks behind the Hubble Telescope), which financially supported Matplotlib's development and greatly expanded its capabilities.

One of Matplotlib's most important features is its ability to play well with many operating systems and graphics backends. Matplotlib supports dozens of backends and output types, which means you can count on it to work regardless of which operating system you are using or which output format you wish. This cross-platform, everything-to-everyone approach has been one of the great strengths of Matplotlib. It has led to a large userbase, which in turn has led to an active developer base and Matplotlib's powerful tools and ubiquity within the scientific Python world.

1. IMPORTING MATPLOTLIB

Before we dive into the details of creating visualizations with Matplotlib, there are a few useful things you should know about using the package.

Visualization with Matplotlib

General Matplotlib Tips

```
In[1]: import matplotlib as mpl  
import matplotlib.pyplot as plt
```

plt.style directive to choose appropriate aesthetic styles for our figures

```
In[2]: plt.style.use('classic')
```

1.1 Plotting from a script

If you are using Matplotlib from within a script, the function `plt.show()` is your friend. `plt.show()` starts an event loop, looks for all currently active figure objects, and opens one or more interactive windows that display your figure or figures. So, for example, you may have a file called *myplot.py* containing the following:

```
# file: myplot.py
```

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)

plt.plot(x, np.sin(x))
plt.plot(x, np.cos(x))

plt.show()
```

You can then run this script from the command-line prompt, which will result in a window opening with your figure displayed:

```
$ python myplot.py
```

The `plt.show` command does a lot under the hood, as it must interact with your system's interactive graphical backend. The details of this operation can vary greatly from system to system and even installation to installation, but Matplotlib does its best to hide all these details from you.

One thing to be aware of: the `plt.show` command should be used *only once* per Python session, and is most often seen at the very end of the script. Multiple show commands can lead to unpredictable backend-dependent behavior, and should mostly be avoided.

1.2 Plotting from an IPython shell

IPython is built to work well with Matplotlib if you specify Matplotlib mode. To enable this mode, you can use the `%matplotlib` magic command after starting ipython:

```
In [1]: %matplotlib
Using matplotlib backend: TkAgg

In [2]: import matplotlib.pyplot as plt
```

At this point, any `plt` plot command will cause a figure window to open, and further commands can be run to update the plot. Some changes (such as modifying properties of lines that are already drawn) will not draw automatically: to force an update, use `plt.draw`. Using `plt.show` in IPython's Matplotlib mode is not required.

1.3 Plotting from an IPython notebook

The Jupyter notebook is a browser-based interactive data analysis tool that can combine narrative, code, graphics, HTML elements, and much more into a single executable document (see IPython: Beyond Normal Python).

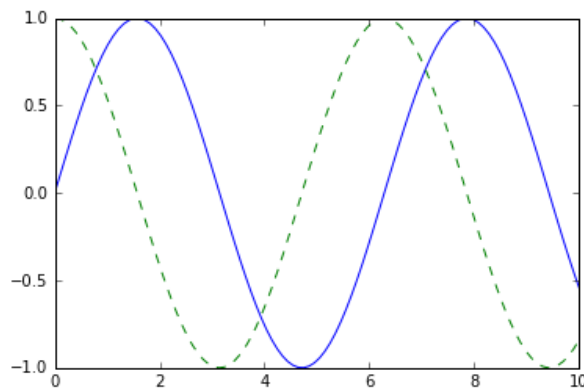
Plotting interactively within a Jupyter notebook can be done with the `%matplotlib` command, and works in a similar way to the IPython shell. You also have the option of embedding graphics directly in the notebook, with two possible options:

- `%matplotlib inline` will lead to *static* images of your plot embedded in the notebook.
- `%matplotlib notebook` will lead to *interactive* plots embedded within the notebook.

For this book, we will generally stick with the default, with figures rendered as static images (see the following figure for the result of this basic plotting example):

```
%matplotlib inline

import numpy as np
x = np.linspace(0, 10, 100)
fig = plt.figure()
plt.plot(x, np.sin(x), '-b')
plt.plot(x, np.cos(x), '-r');
```



1.4 Saving Figures to File

One nice feature of Matplotlib is the ability to save figures in a wide variety of formats. Saving a figure can be done using the `savefig` command. For example, to save the previous figure as a PNG file, we can run this:

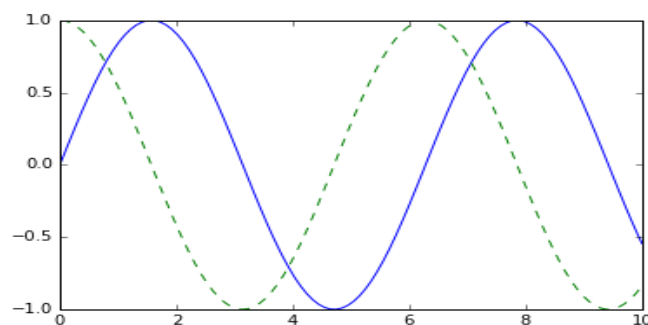
```
In[5]: fig.savefig('my_figure.png')
```

Save figure as png

```
IN [6] ls -lh my_figure.png # shows figure properties, ls list file, -lh – human readable format,
```

```
Out [6]: -rw-r--r-- 1 jakevdp staff 16K Aug 11 10:59 my_figure.png
```

```
In[7]: from IPython.display import Image
        Image('my_figure.png')
```



```
In[8]: fig.canvas.get_supported_filetypes() # list supported file format to save figures
```

```
Out[8]: {'eps': 'Encapsulated Postscript',
```

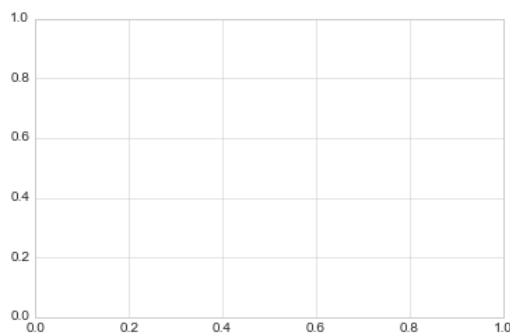
```
'jpeg': 'Joint Photographic Experts Group',  
'jpg': 'Joint Photographic Experts Group',  
'pdf': 'Portable Document Format',  
'pgf': 'PGF code for LaTeX',  
'png': 'Portable Network Graphics',  
'ps': 'Postscript',  
'raw': 'Raw RGBA bitmap',  
'rgba': 'Raw RGBA bitmap',  
'svg': 'Scalable Vector Graphics',  
'svgz': 'Scalable Vector Graphics',  
'tif': 'Tagged Image File Format',  
'tiff': 'Tagged Image File Format'}
```

2. SIMPLE LINE PLOTS

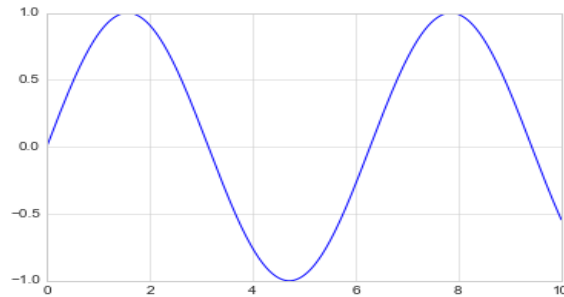
Perhaps the simplest of all plots is the visualization of a single function $y=f(x)$. Here we will take a first look at creating a simple plot of this type. As in all the following chapters, we'll start by setting up the notebook for plotting and importing the packages we will use:

```
In[1]: %matplotlib inline  
import matplotlib.pyplot as plt  
plt.style.use('seaborn-whitegrid')  
import numpy as np
```

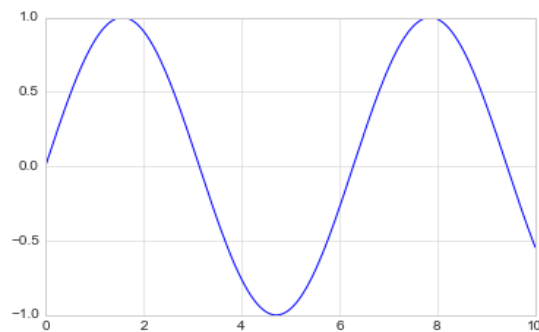
```
In[2]: fig = plt.figure()    #creates figure  
ax = plt.axes()             #creates axes
```



```
In[3]: fig = plt.figure()  
ax = plt.axes()  
x = np.linspace(0, 10, 1000)  
# start, stop, no. of points.  
# Return evenly spaced numbers over a specified interval. will also work without 1000  
ax.plot(x, np.sin(x));
```

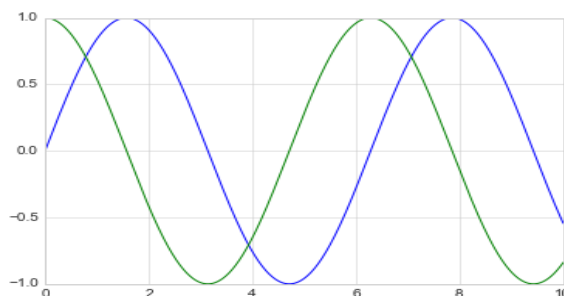


```
In[4]: plt.plot(x, np.sin(x));
```



```
In[5]: plt.plot(x, np.sin(x))
        plt.plot(x, np.cos(x));
```

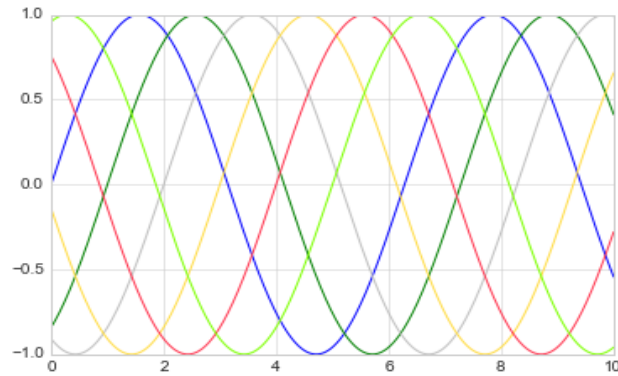
If we want to create a single figure with multiple lines (see the following figure), we can simply call the plot function multiple times:



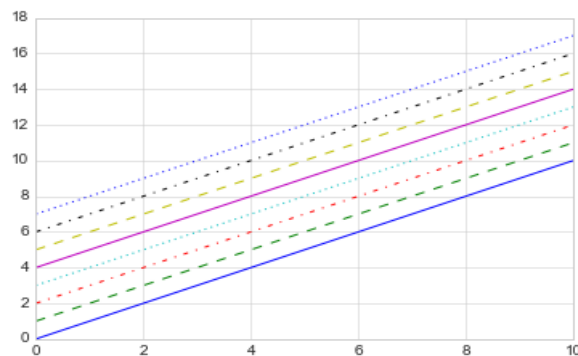
2.1 Adjusting the Plot: Line Colors and Styles

The first adjustment you might wish to make to a plot is to control the line colors and styles. The `plt.plot` function takes additional arguments that can be used to specify these. To adjust the color, you can use the `color` keyword, which accepts a string argument representing virtually any imaginable color. The color can be specified in a variety of ways; see the following figure for the output of the following examples:

```
In[6]: plt.plot(x, np.sin(x - 0), color='blue') # specify color by name
        plt.plot(x, np.sin(x - 1), color='g') # short color code (rgbcmk)
        plt.plot(x, np.sin(x - 2), color='0.75') # Grayscale between 0 and 1
        plt.plot(x, np.sin(x - 3), color='#FFDD44') # Hex code (RRGGBB from 00 to FF)
        plt.plot(x, np.sin(x - 4), color=(1.0,0.2,0.3)) # RGB tuple, values 0 and 1
        plt.plot(x, np.sin(x - 5), color='chartreuse'); # all HTML color names supported
```

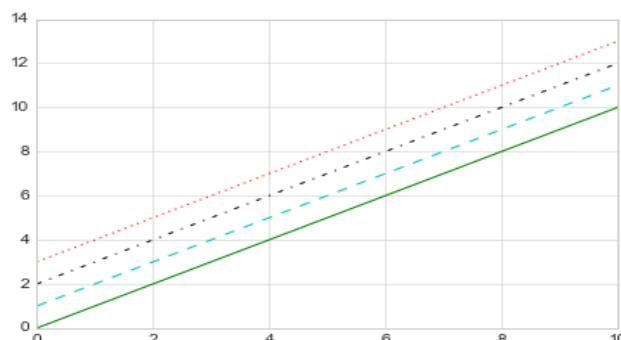


```
In[7]: plt.plot(x, x + 0, linestyle='solid')
      plt.plot(x, x + 1, linestyle='dashed')
      plt.plot(x, x + 2, linestyle='dashdot')
      plt.plot(x, x + 3, linestyle='dotted');
# For short, you can use the following codes:
      plt.plot(x, x + 4, linestyle='-') # solid
      plt.plot(x, x + 5, linestyle='--') # dashed
      plt.plot(x, x + 6, linestyle='-.') # dashdot
      plt.plot(x, x + 7, linestyle=':'); # dotted
```



Though it may be less clear to someone reading your code, you can save some keystrokes by combining these linestyle and color codes into a single non-keyword argument to the `plt.plot` function; the following figure shows the result:

```
In[8]: plt.plot(x, x + 0, '-g') # solid green
      plt.plot(x, x + 1, '--c') # dashed cyan
      plt.plot(x, x + 2, '-.k') # dashdot black
      plt.plot(x, x + 3, ':r'); # dotted red
```

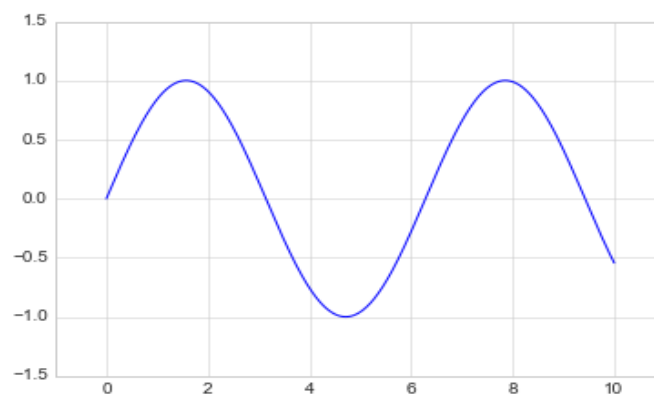


These single-character color codes reflect the standard abbreviations in the RGB (Red/Green/Blue) and CMYK (Cyan/Magenta/Yellow/black) color systems, commonly used for digital color graphics.

2.2 Adjusting the Plot: Axes Limits

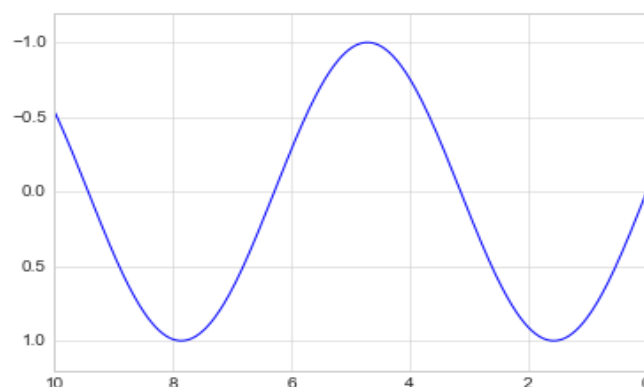
Matplotlib does a decent job of choosing default axes limits for your plot, but sometimes it's nice to have finer control. The most basic way to adjust the limits is to use the `plt.xlim` and `plt.ylim` functions (see the following figure):

```
plt.plot(x, np.sin(x))  
  
plt.xlim(-1, 11)  
plt.ylim(-1.5, 1.5);
```



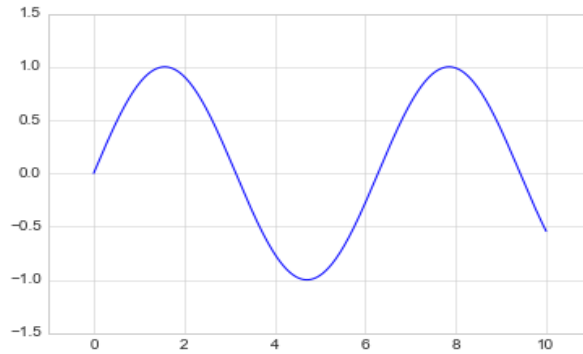
If for some reason you'd like either axis to be displayed in reverse, you can simply reverse the order of the arguments.

```
In[10]: plt.plot(x, np.sin(x))  
        plt.xlim(10, 0)  
        plt.ylim(1.2, -1.2);
```



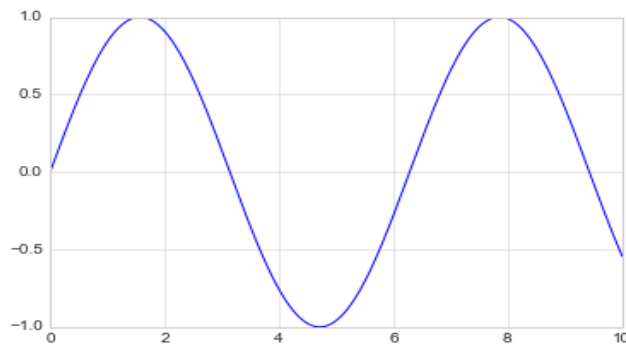
A useful related method is `plt.axis` (note here the potential confusion between *axes* with an *e*, and *axis* with an *i*), which allows more qualitative specifications of axis limits. For example, you can automatically tighten the bounds around the current content, as shown in the following figure:

```
In[11]: plt.plot(x, np.sin(x))  
        plt.axis([-1, 11, -1.5, 1.5]); # x axis limit -1 to 11
```

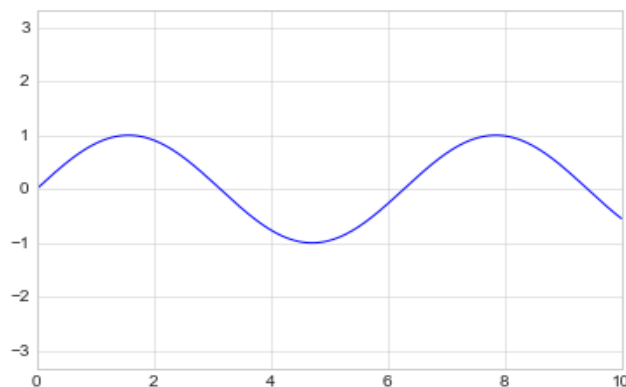


Or you can specify that you want an equal axis ratio, such that one unit in x is visually equivalent to one unit in y, as seen in the following figure:

```
In[12]: plt.plot(x, np.sin(x))  
        plt.axis('tight'); # Frame is tight fitted
```



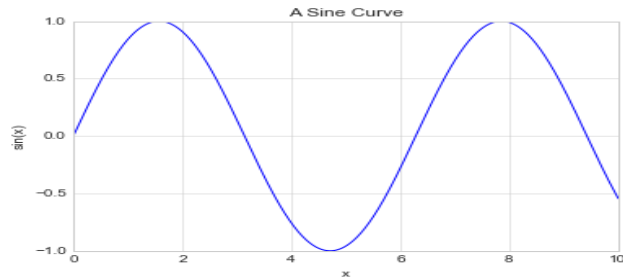
```
In[13]: plt.plot(x, np.sin(x))  
        plt.axis('equal');
```



2.3 Labeling Plots

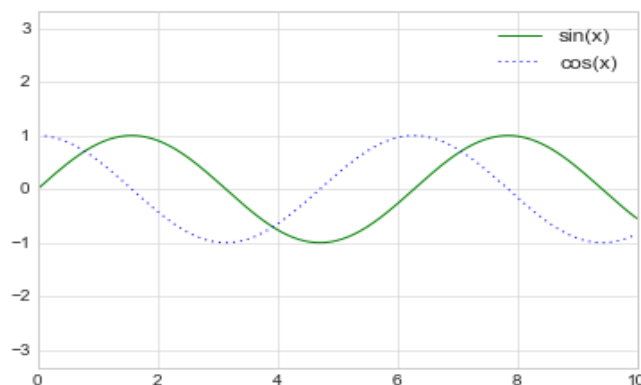
As the last piece of this chapter, we'll briefly look at the labeling of plots: titles, axis labels, and simple legends. Titles and axis labels are the simplest such labels—there are methods that can be used to quickly set them (see the following figure):

```
In[14]: plt.plot(x, np.sin(x))  
        plt.title("A Sine Curve")  
        plt.xlabel("x")  
        plt.ylabel("sin(x)");
```



The position, size, and style of these labels can be adjusted using optional arguments to the functions, described in the docstrings. When multiple lines are being shown within a single axes, it can be useful to create a plot legend that labels each line type. Again, Matplotlib has a built-in way of quickly creating such a legend; it is done via the (you guessed it) `plt.legend` method. Though there are several valid ways of using this, I find it easiest to specify the label of each line using the `label` keyword of the plot function (see the following figure):

```
In[15]:plt.plot(x, np.sin(x), '-g', label='sin(x)')    # green label sin
        plt.plot(x, np.cos(x), ':b', label='cos(x)')  # blue label cos
        plt.axis('equal')
        plt.legend();
```



3. SIMPLE SCATTER PLOTS

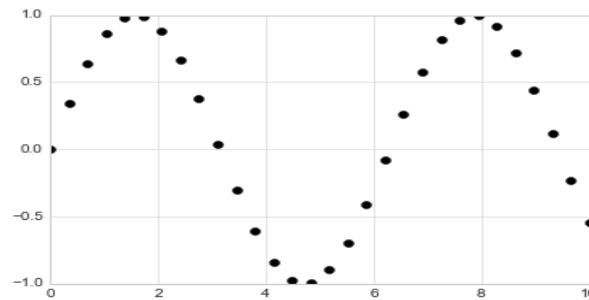
Another commonly used plot type is the simple scatter plot, a close cousin of the line plot. Instead of points being joined by line segments, here the points are represented individually with a dot, circle, or other shape. We'll start by setting up the notebook for plotting and importing the packages we will use:

```
In[1]: %matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('seaborn-whitegrid')
import numpy as np
```

3.1 Scatter Plots with plt.plot

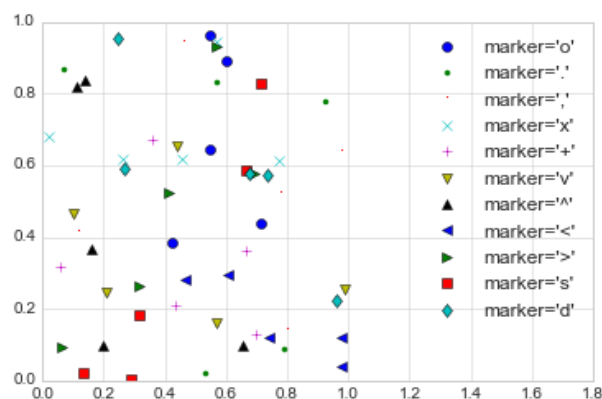
In the previous chapter we looked at using `plt.plot/ax.plot` to produce line plots. It turns out that this same function can produce scatter plots as well (see the following figure):

```
In[2]: x = np.linspace(0, 10, 30)
       y = np.sin(x)
       plt.plot(x, y, 'o', color='black');
```



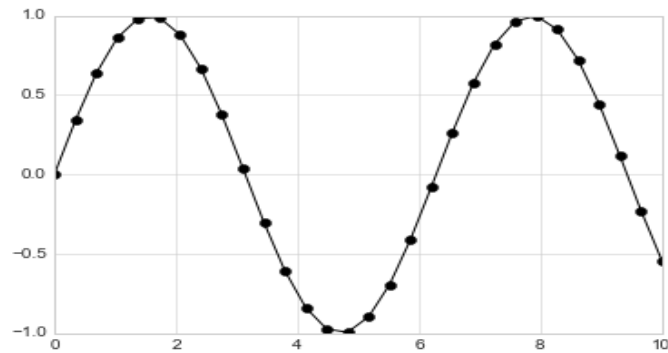
The third argument in the function call is a character that represents the type of symbol used for the plotting. Just as you can specify options such as '-' or '--' to control the line style, the marker style has its own set of short string codes. The full list of available symbols can be seen in the documentation of `plt.plot`, or in Matplotlib's online documentation. Most of the possibilities are fairly intuitive, and a number of the more common ones are demonstrated here (see the following figure)

```
rng = np.random.default_rng(0)
for marker in ['o', '.', ',', 'x', '+', 'v', '^', '<', '>', 's', 'd']:
    plt.plot(rng.random(2), rng.random(2), marker, color='black',
             label="marker='{0}'".format(marker))
plt.legend(numpoints=1, fontsize=13)
plt.xlim(0, 1.8);
```

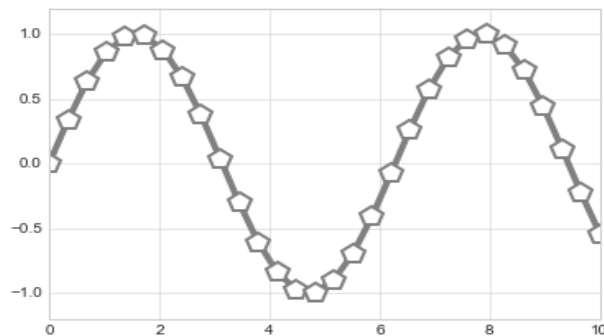


For even more possibilities, these character codes can be used together with line and color codes to plot points along with a line connecting them (see the following figure):

```
In[4]: plt.plot(x, y, '-ok'); # line (-), circle marker (o), black (k)
```



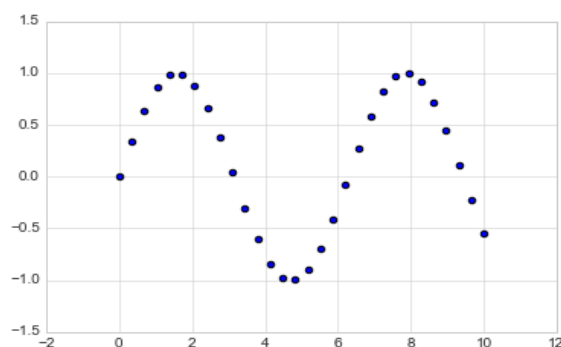
```
In[5]: plt.plot(x, y, '-p', color='gray', # -p pentagon
               markersize=15, linewidth=4,
               markerfacecolor='white',
               markeredgecolor='gray',
               markeredgewidth=2)
plt.ylim(-1.2, 1.2);
```



3.2 Scatter Plots with plt.scatter

A second, more powerful method of creating scatter plots is the `plt.scatter` function, which can be used very similarly to the `plt.plot` function (see the following figure):

```
In[6]: plt.scatter(x, y, marker='o');
```



The primary difference of `plt.scatter` from `plt.plot` is that it can be used to create scatter plots where the properties of each individual point (size, face color, edge color, etc.) can be individually controlled or mapped to data.

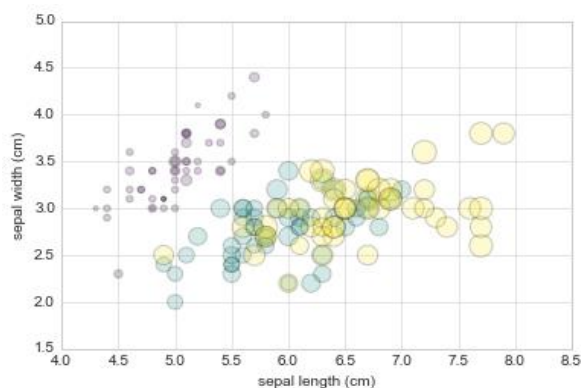
Let's show this by creating a random scatter plot with points of many colors and sizes. In order to better see the overlapping results, we'll also use the alpha keyword to adjust the transparency level (see the following figure):

```
In[7]: rng = np.random.RandomState(0)
       x = rng.randn(100)
       y = rng.randn(100)
       colors = rng.rand(100)
       sizes = 1000 * rng.rand(100)
       plt.scatter(x, y, c=colors, s=sizes, alpha=0.3, cmap='viridis')
       plt.colorbar(); # show color scale
```



Notice that the color argument is automatically mapped to a color scale (shown here by the colorbar command), and that the size argument is given in pixels. In this way, the color and size of points can be used to convey information in the visualization, in order to visualize multidimensional data.

```
In[8]: from sklearn.datasets import load_iris
       iris = load_iris()
       features = iris.data.T #Transpose
       plt.scatter(features[0], features[1], alpha=0.2, s=100*features[3], c=iris.target,
       cmap='viridis')
       plt.xlabel(iris.feature_names[0])
       plt.ylabel(iris.feature_names[1]);
```



4. VISUALIZING ERRORS

For any scientific measurement, accurate accounting of uncertainties is nearly as important, if not more so, as accurate reporting of the number itself. For example, imagine that

I am using some astrophysical observations to estimate the Hubble Constant, the local measurement of the expansion rate of the Universe. I know that the current literature suggests a value of around 70 (km/s)/Mpc, and I measure a value of 74 (km/s)/Mpc with my method. Are the values consistent? The only correct answer, given this information, is this: there is no way to know.

Suppose I augment this information with reported uncertainties: the current literature suggests a value of 70 ± 2.5 (km/s)/Mpc, and my method has measured a value of 74 ± 5 (km/s)/Mpc. Now are the values consistent? That is a question that can be quantitatively answered. In visualization of data and results, showing these errors effectively can make a plot convey much more complete information.

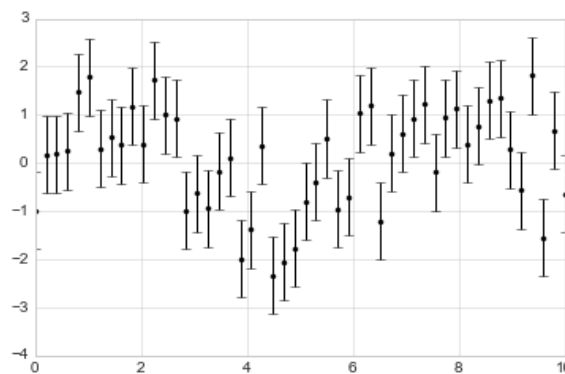
Basic Errorbars

One standard way to visualize uncertainties is using an errorbar. A basic errorbar can be created with a single Matplotlib function call, as shown in the following figure:

```
In[1]: %matplotlib inline

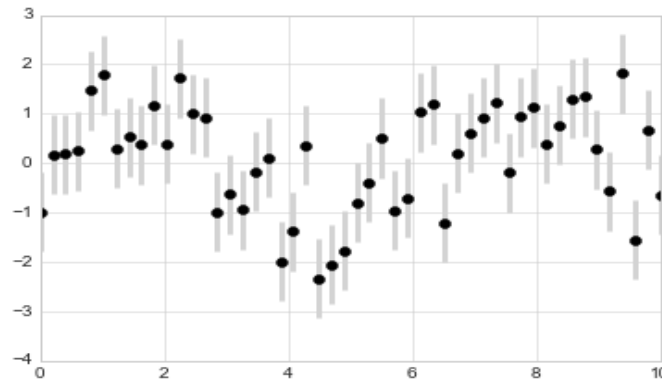
import matplotlib.pyplot as plt
plt.style.use('seaborn-whitegrid')
import numpy as np

In[2]: x = np.linspace(0, 10, 50)
       dy = 0.8
       y = np.sin(x) + dy * np.random.randn(50)
       plt.errorbar(x, y, yerr=dy, fmt='k');
```



Here the `fmt` is a format code controlling the appearance of lines and points, and it has the same syntax as the shorthand used in `plt.plot`, outlined in the previous chapter and earlier in this chapter. In addition to these basic options, the `errorbar` function has many options to fine-tune the outputs. Using these additional options, you can easily customize the aesthetics of your errorbar plot. I often find it helpful, especially in crowded plots, to make the errorbars lighter than the points themselves.

```
In[3]: plt.errorbar(x, y, yerr=dy, fmt='o', color='black', ecolor='lightgray', elinewidth=3,
                  capsize=0);
```



In addition to these options, you can also specify horizontal errorbars, one-sided errorbars, and many other variants. For more information on the options available, refer to the docstring of `plt.errorbar`.

4.1 Continuous Errors

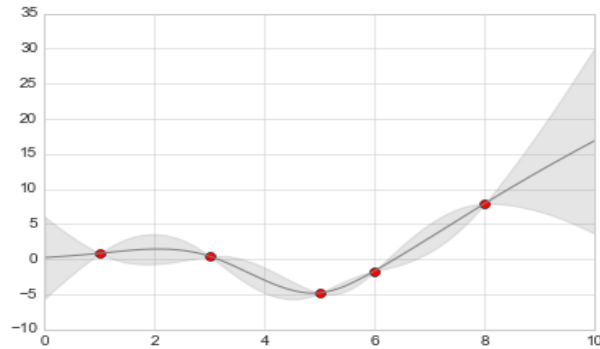
In some situations, it is desirable to show errorbars on continuous quantities. Though Matplotlib does not have a built-in convenience routine for this type of application, it's relatively easy to combine primitives like `plt.plot` and `plt.fill_between` for a useful result.

Here we'll perform a simple *Gaussian process regression*, using the Scikit-Learn API (see *Introducing Scikit-Learn* for details). This is a method of fitting a very flexible nonparametric function to data with a continuous measure of the uncertainty. We won't delve into the details of Gaussian process regression at this point, but will focus instead on how you might visualize such a continuous error measurement:

```

In[4]: from sklearn.gaussian_process import GaussianProcess
        # define the model and draw some data
        model = lambda x: x * np.sin(x)
        xdata = np.array([1, 3, 5, 6, 8])
        ydata = model(xdata)
        # Compute the Gaussian process fit
        # cubic correlation function
        gp = GaussianProcess(corr='cubic', theta0=1e-2, - thetaL=1e 4, thetaU=1E-1,
                             random_start=100)
        gp.fit(xdata[:, np.newaxis], ydata)
        xfit = np.linspace(0, 10, 1000)
        yfit, MSE = gp.predict(xfit[:, np.newaxis], eval_MSE=True)
        dyfit = 2 * np.sqrt(MSE) # 2*sigma ~ 95% confidence region

In[5]: # Visualize the result
        plt.plot(xdata, ydata, 'or')
        plt.plot(xfit, yfit, '-', color='gray')
        plt.fill_between(xfit, yfit - dyfit, yfit + dyfit,
                         color='gray', alpha=0.2)
        plt.xlim(0, 10);
  
```



Take a look at the `fill_between` call signature: we pass an `x` value, then the lower `y`-bound, then the upper `y`-bound, and the result is that the area between these regions is filled.

5. DENSITY AND CONTOUR PLOTS

Sometimes it is useful to display three-dimensional data in two dimensions using contours or color-coded regions. There are three Matplotlib functions that can be helpful for this task: `plt.contour` for contour plots, `plt.contourf` for filled contour plots, and `plt.imshow` for showing images. This chapter looks at several examples of using these. We'll start by setting up the notebook for plotting and importing the functions we will use:

```
In[1]: %matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('seaborn-white')
import numpy as np
```

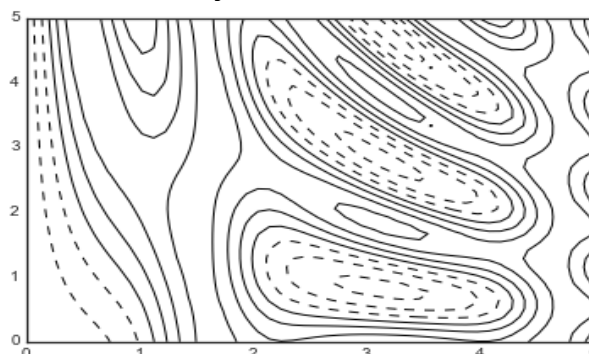
5.1 Visualizing a Three-Dimensional Function

Our first example demonstrates a contour plot using a function $z=f(x,y)$, using the following particular choice for f (we've seen this before in Computation on Arrays: Broadcasting, when we used it as a motivating example for array broadcasting)

```
In[2]: def f(x, y):
return np.sin(x) ** 10 + np.cos(10 + y * x) * np.cos(x)

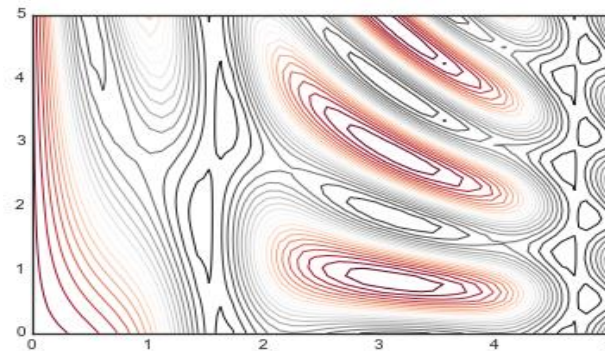
In[3]: x = np.linspace(0, 5, 50)
y = np.linspace(0, 5, 40)
X, Y = np.meshgrid(x, y) # Return coordinate matrices from coordinate vectors
Z = f(X, Y)

In[4]: plt.contour(X, Y, Z, colors='black');
```



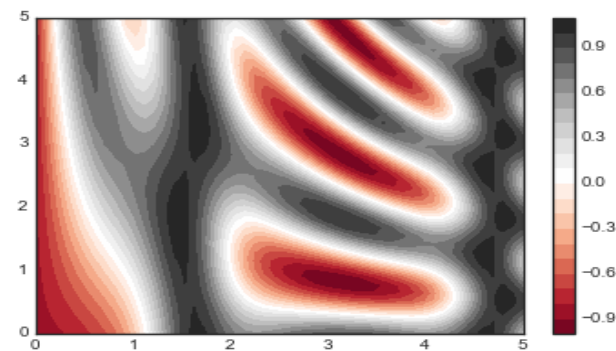
Notice that when a single color is used, negative values are represented by dashed lines and positive values by solid lines. Alternatively, the lines can be color-coded by specifying a colormap with the `cmap` argument. Here we'll also specify that we want more lines to be drawn, at 20 equally spaced intervals within the data range, as shown in the following figure:

```
In[5]: plt.contour(X, Y, Z, 20, cmap='RdGy'); # 20 - contour levels
```



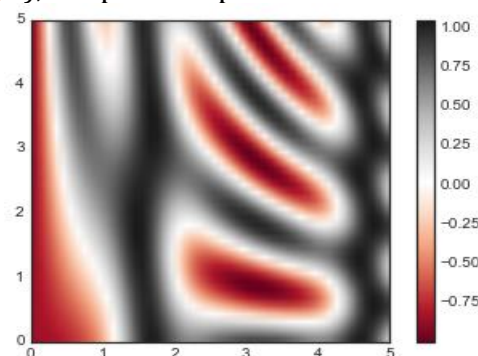
Here we chose the RdGy (short for *Red-Gray*) colormap, which is a good choice for divergent data: (i.e., data with positive and negative variation around zero). Matplotlib has a wide range of colormaps available, which you can easily browse in IPython by doing a tab completion on the `plt.cm` module:

```
In[6]: plt.contourf(X, Y, Z, 20, cmap='RdGy') # 20 - contour levels
plt.colorbar();
```

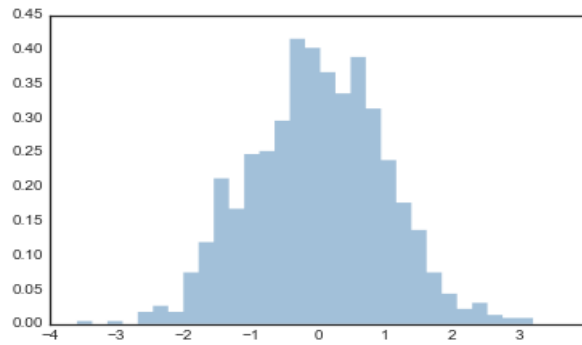


The colorbar makes it clear that the black regions are "peaks," while the red regions are "valleys." One potential issue with this plot is that it is a bit splotchy: the color steps are discrete rather than continuous, which is not always what is desired. This could be remedied by setting the number of contours to a very high number, but this results in a rather inefficient plot: Matplotlib must render a new polygon for each step in the level. A better way to generate a smooth representation is to use the `plt.imshow` function, which offers the interpolation argument to generate a smooth two-dimensional representation of the data (see the following figure):

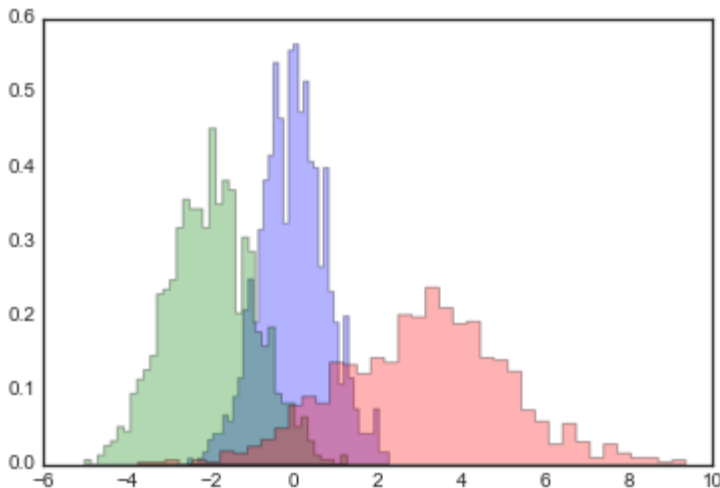
```
In[7]: plt.imshow(Z, extent=[0, 5, 0, 5], origin='lower', cmap='RdGy')
plt.colorbar()
plt.axis(aspect='image'); #aspect - aspect ratio
```



```
In[3]: plt.hist(data, bins=30, normed=True, alpha=0.5, histtype='stepfilled',
color='steelblue',
edgecolor='none'); # bins - bar, normed - histogram is normalized,
histtype - generates a lineplot that is by default filled.
histtype{'bar', 'barstacked', 'step', 'stepfilled'}, default: 'bar'
```



```
In[4]: x1 = np.random.normal(0, 0.8, 1000)
      x2 = np.random.normal(-2, 1, 1000)
      x3 = np.random.normal(3, 2, 1000)
      kwargs = dict(histtype='stepfilled', alpha=0.3, normed=True, bins=40)
      plt.hist(x1, **kwargs) #**kwargs in function definitions in python is used to pass a
      keyworded, variable-length argument list
      plt.hist(x2, **kwargs)
      plt.hist(x3, **kwargs);
```



If you are interested in computing, but not displaying, the histogram (that is, counting the number of points in a given bin), you can use the `np.histogram` function:

```
In[5]: counts, bin_edges = np.histogram(data, bins=5) # bin_edges - contain edges of bin
      print(counts)
      [ 12 190 468 301 29]
```

6.1 Two-Dimensional Histograms and Binnings

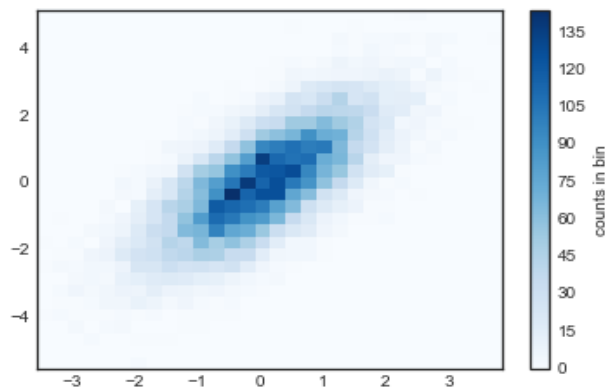
Just as we create histograms in one dimension by dividing the number line into bins, we can also create histograms in two dimensions by dividing points among two-dimensional bins. We'll take a brief look at several ways to do this here. We'll start by defining some data—an `x` and `y` array drawn from a multivariate Gaussian distribution:

```
In[6]: mean = [0, 0]
      cov = [[1, 1], [1, 2]]
      x, y = np.random.multivariate_normal(mean, cov, 10000).T #generate samples from
      multivariate normal distribution
```

6.1.1 plt.hist2d: Two-dimensional histogram

One straightforward way to plot a two-dimensional histogram is to use Matplotlib's `plt.hist2d` function (see the following figure):

```
In[12]: plt.hist2d(x, y, bins=30, cmap='Blues')
        cb = plt.colorbar()
        cb.set_label('counts in bin')
```

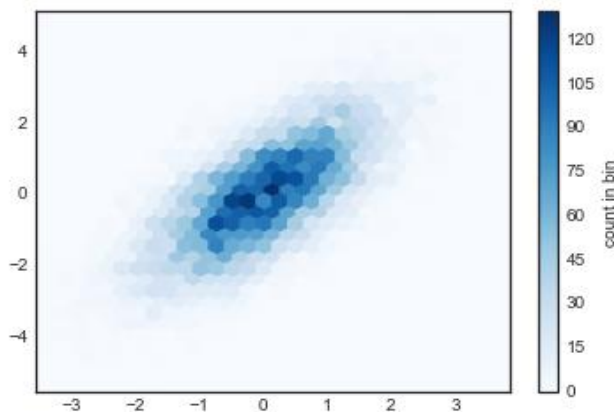


```
In[8]: counts, xedges, yedges = np.histogram2d(x, y, bins=30) # return values count,x,y
```

6.1.2 plt.hexbin: Hexagonal binnings

The two-dimensional histogram creates a tessellation of squares across the axes. Another natural shape for such a tessellation is the regular hexagon. For this purpose, Matplotlib provides the `plt.hexbin` routine, which represents a two-dimensional dataset binned within a grid of hexagons (see the following figure):

```
In[9]: plt.hexbin(x, y, gridsize=30, cmap='Blues')
        cb = plt.colorbar(label='count in bin')
```



6.1.3 Kernel density estimation.

- Another common method of evaluating densities in multiple dimensions is kernel density estimation (KDE).
- KDE can be thought of as a way to “smear out” the points in space and add up the result to obtain a smooth function.

```
In[10]: from scipy.stats import gaussian_kde
        # fit an array of size [Ndim, Nsamples]
        data = np.vstack([x, y])
        kde = gaussian_kde(data)

        # evaluate on a regular grid
        xgrid = np.linspace(-3.5, 3.5, 40)
```

```

ygrid = np.linspace(-6, 6, 40)
Xgrid, Ygrid = np.meshgrid(xgrid, ygrid)
Z = kde.evaluate(np.vstack([Xgrid.ravel(), Ygrid.ravel()])) # returns 1D array

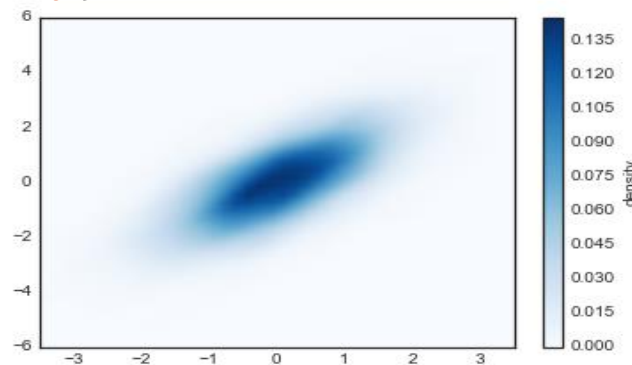
```

Plot the result as an image

```

plt.imshow(Z.reshape(Xgrid.shape), origin='lower', aspect='auto', extent=[-3.5, 3.5, -6, 6], cmap='Blues') # reshapes 1D Z to 2D #aspect ratio # x and y axis # color map
cb = plt.colorbar()
cb.set_label("density")

```



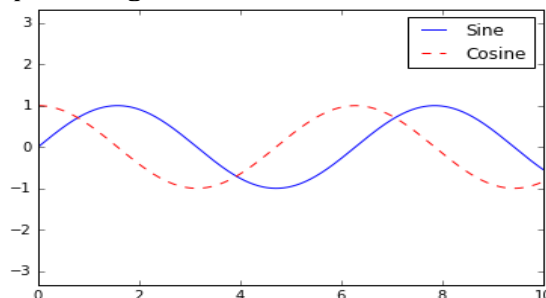
7. CUSTOMIZING PLOT LEGENDS

Plot legends give meaning to a visualization, assigning meaning to the various plot elements. We previously saw how to create a simple legend; here we'll take a look at customizing the placement and aesthetics of the legend in Matplotlib. The simplest legend can be created with the `plt.legend` command, which automatically creates a legend for any labeled plot elements

```

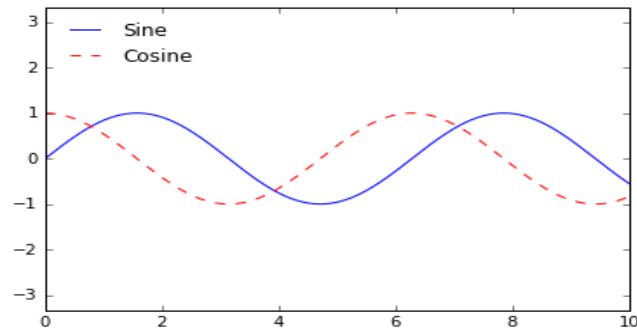
In[1]: import matplotlib.pyplot as plt
        plt.style.use('classic')
In[2]: %matplotlib inline
        import numpy as np
In[3]: x = np.linspace(0, 10, 1000)
        fig, ax = plt.subplots() # create a figure and a set of subplots.
        ax.plot(x, np.sin(x), '-b', label='Sine')
        ax.plot(x, np.cos(x), '--r', label='Cosine')
        ax.axis('equal')
        leg = ax.legend(); # prints legend

```



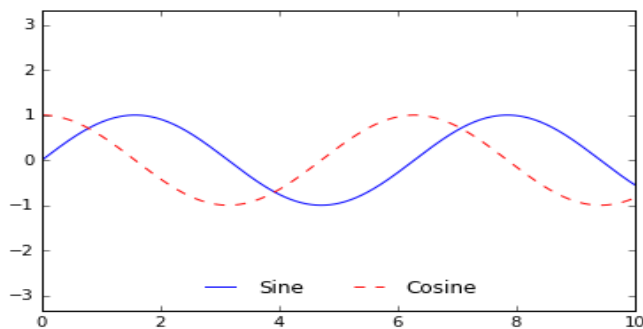
```
In[4]: ax.legend(loc='upper left', frameon=False)
```

Fig



```
In[5]: ax.legend(frameon=False, loc='lower center', ncol=2)
```

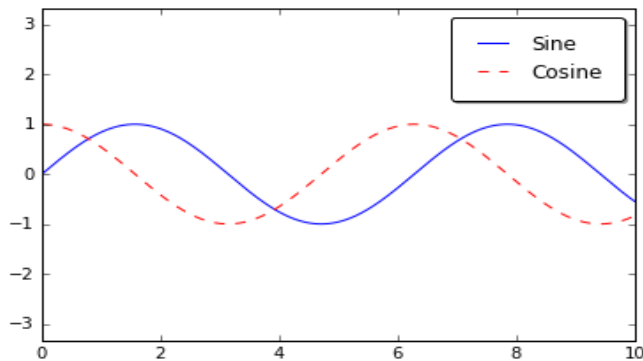
Fig



```
In[6]: ax.legend(fancybox=True, framealpha=1, shadow=True, borderpad=1)
```

frame alpha – type of box frame

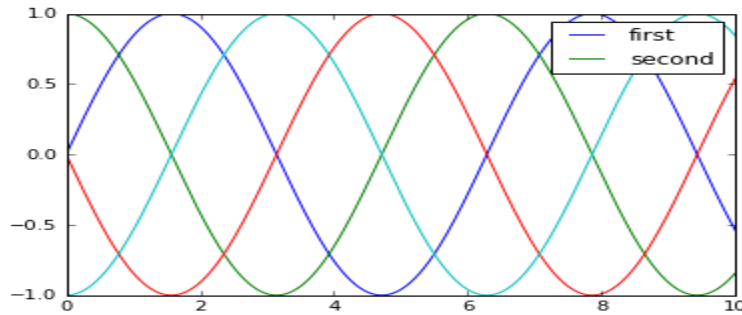
Fig



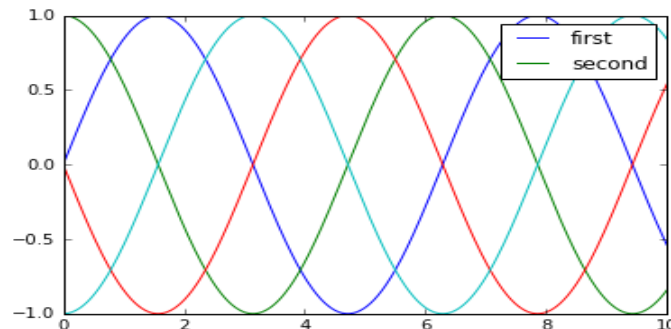
7.1 Choosing Elements for the Legend

As we have already seen, by default the legend includes all labeled elements from the plot. If this is not what is desired, we can fine-tune which elements and labels appear in the legend by using the objects returned by plot commands. `plt.plot` is able to create multiple lines at once, and returns a list of created line instances. Passing any of these to `plt.legend` will tell it which to identify, along with the labels we'd like to specify.

```
In[7]: y = np.sin(x[:, np.newaxis] + np.pi * np.arange(0, 2, 0.5)
      lines = plt.plot(x, y)
      plt.legend(lines[:2], ['first', 'second']);
```



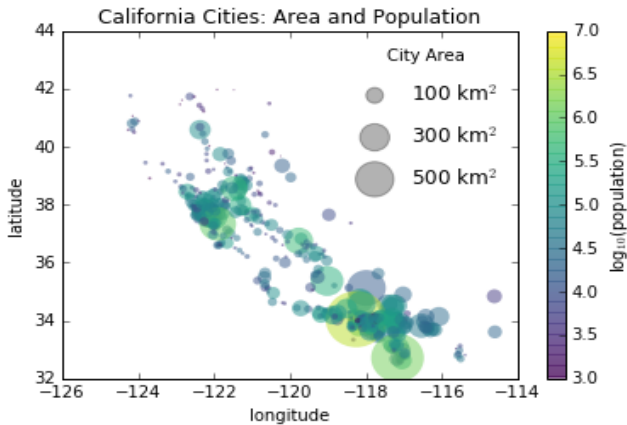
```
In[8]: plt.plot(x, y[:, 0], label='first')
plt.plot(x, y[:, 1], label='second')
plt.plot(x, y[:, 2:]) # x with y - all columns from the third column onward
plt.legend(framealpha=1, frameon=True);
```



7.2 Legend for Size of Points

Sometimes the legend defaults are not sufficient for the given visualization. For example, perhaps you're using the size of points to mark certain features of the data, and want to create a legend reflecting this. Here is an example where we'll use the size of points to indicate populations of California cities. We'd like a legend that specifies the scale of the sizes of the points, and we'll accomplish this by plotting some labeled data with no entries.

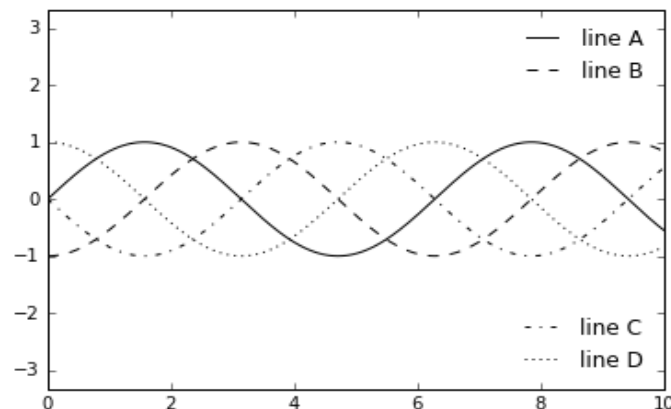
```
In[9]: import pandas as pd
cities = pd.read_csv('data/california_cities.csv')
# Extract the data we're interested in
lat, lon = cities['latd'], cities['longd']
population, area = cities['population_total'], cities['area_total_km2']
# Scatter the points, using size and color but no label
plt.scatter(lon, lat, label=None, c=np.log10(population), cmap='viridis',
s=area, linewidth=0, alpha=0.5)
plt.axis(aspect='equal')
plt.xlabel('longitude')
plt.ylabel('latitude')
plt.colorbar(label='log$_{10}$(population)')
plt.clim(3, 7) # Set the color limits of the current image. 3- lower, 7 - upper
# Here we create a legend:
# we'll plot empty lists with the desired size and label
for area in [100, 300, 500]: #For area in values of 100, 300, 500
plt.scatter([], [], c='k', alpha=0.3, s=area, label=str(area) + ' km$^2$')
plt.legend(scatterpoints=1, frameon=False, labelspace=1, title='City Area')
plt.title('California Cities: Area and Population');
```



7.3 Multiple Legends

Sometimes when designing a plot you'd like to add multiple legends to the same axes. Unfortunately, Matplotlib does not make this easy: via the standard legend interface, it is only possible to create a single legend for the entire plot. If you try to create a second legend using `plt.legend` or `ax.legend`, it will simply override the first one. We can work around this by creating a new legend artist from scratch (Artist is the base class Matplotlib uses for visual attributes), and then using the lower-level `ax.add_artist` method to manually add the second artist to the plot

```
In[10]: fig, ax = plt.subplots()
        lines = []
        styles = ['-', '--', '-.', ':']
        x = np.linspace(0, 10, 1000)
        for i in range(4):
            lines += ax.plot(x, np.sin(x - i * np.pi / 2), styles[i], color='black')
            ax.axis('equal')
            # specify the lines and labels of the first legend
            ax.legend(lines[:2], ['line A', 'line B'],
                    loc='upper right', frameon=False)
            # Create the second legend and add the artist manually.
            from matplotlib.legend import Legend
            leg = Legend(ax, lines[2:], ['line C', 'line D'],
                    loc='lower right', frameon=False)
            ax.add_artist(leg);
```

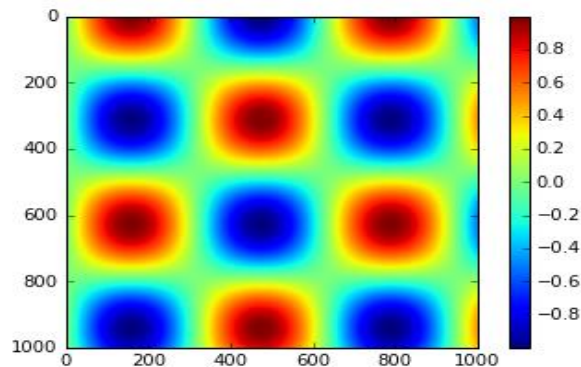


8. CUSTOMIZING COLORBARS

Plot legends identify discrete labels of discrete points. For continuous labels based on the color of points, lines, or regions, a labeled colorbar can be a great tool. In Matplotlib, a colorbar is drawn as a separate axes that can provide a key for the meaning of colors in a plot. Because the book is printed in black and white, this chapter has an accompanying online supplement where you can view the figures in full color. We'll start by setting up the notebook for plotting and importing the functions we will use

```

In[1]: import matplotlib.pyplot as plt
        plt.style.use('classic')
In[2]: %matplotlib inline
        import numpy as np
In[3]: x = np.linspace(0, 10, 1000)
        I = np.sin(x) * np.cos(x[:, np.newaxis])
        plt.imshow(I)
        plt.colorbar();
  
```

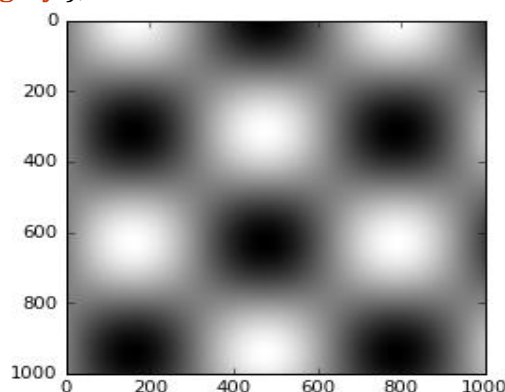


8.1 Customizing Colorbars

The colormap can be specified using the `cmap` argument to the plotting function that is creating the visualization.

```

In[4]: plt.imshow(I, cmap='gray');
  
```



Choosing the colormap

- **Sequential colormaps**

These consist of one continuous sequence of colors (e.g., binary or viridis).

- **Divergent colormaps**

These usually contain two distinct colors, which show positive and negative deviations from a mean (e.g., RdBu or PuOr).

- **Qualitative colormaps**

These mix colors with no particular sequence (e.g., rainbow or jet).

```
from matplotlib.colors import LinearSegmentedColormap
```

```
def grayscale_cmap(cmap):
    """Return a grayscale version of the given colormap"""
    cmap = plt.cm.get_cmap(cmap)
    colors = cmap(np.arange(cmap.N))

    # convert RGBA to perceived grayscale luminance
    # cf. http://alienryderflex.com/hsp.html
    RGB_weight = [0.299, 0.587, 0.114]
    luminance = np.sqrt(np.dot(colors[:, :3] ** 2, RGB_weight))
    colors[:, :3] = luminance[:, np.newaxis]    #color store rgb values,

    return LinearSegmentedColormap.from_list(cmap.name + "_gray", colors, cmap.N)

def view_colormap(cmap):
    """Plot a colormap with its grayscale equivalent"""
    cmap = plt.cm.get_cmap(cmap)
    colors = cmap(np.arange(cmap.N))

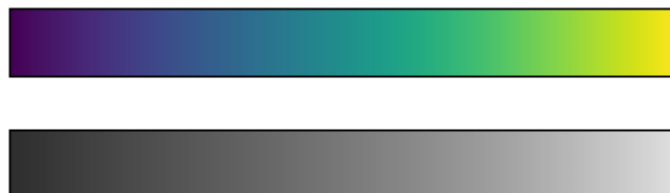
    cmap = grayscale_cmap(cmap)
    grayscale = cmap(np.arange(cmap.N))

    fig, ax = plt.subplots(2, figsize=(6, 2), #size of figure
        subplot_kw=dict(xticks=[], yticks=[])) # ticks are empty
    ax[0].imshow([colors], extent=[0, 10, 0, 1]) # extent image boundary
    ax[1].imshow([grayscale], extent=[0, 10, 0, 1]) # limits of x and y axis
```

```
In[6]: view_colormap('jet')
```



```
In[7]: view_colormap('viridis')
```



```
In[8]: view_colormap('cubehelix')
```



In[9]: `view_colormap('RdBu')`



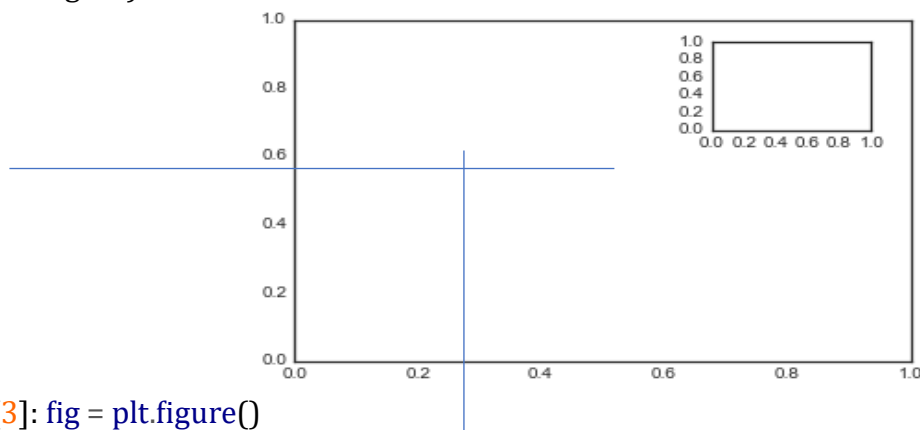
9. Multiple Subplots

Sometimes it is helpful to compare different views of data side by side. To this end, Matplotlib has the concept of *subplots*: groups of smaller axes that can exist together within a single figure. These subplots might be insets, grids of plots, or other more complicated layouts. In this chapter we'll explore four routines for creating subplots in Matplotlib.

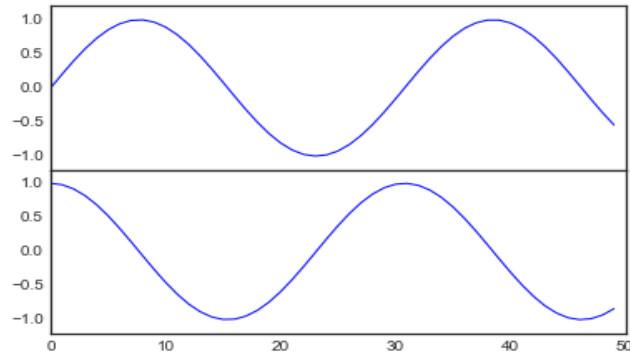
```
In[1]: %matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('seaborn-white')
import numpy as np
```

```
In[2]: ax1 = plt.axes() # standard axes
ax2 = plt.axes([0.65, 0.65, 0.2, 0.2]) # position, size of subplot
```

Create an inset axes at the top-right corner of another axes by setting the x and y position to 0.65 (that is, starting at 65% of the width and 65% of the height of the figure) and the x and y extents to 0.2 (that is, the size of the axes is 20% of the width and 20% of the height of the figure).



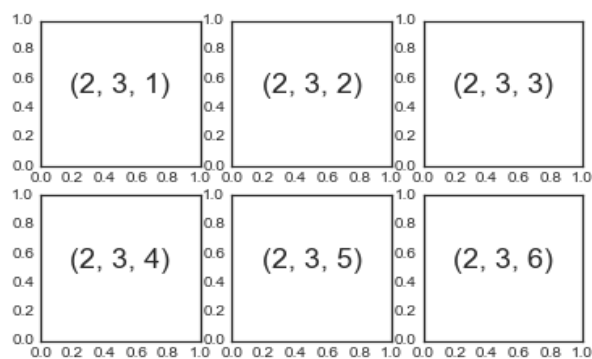
```
In[3]: fig = plt.figure()
ax1 = fig.add_axes([0.1, 0.5, 0.8, 0.4], # position and size of the subplot within the
figure.
xticklabels=[], ylim=(-1.2, 1.2))
ax2 = fig.add_axes([0.1, 0.1, 0.8, 0.4],
ylim=(-1.2, 1.2))
x = np.linspace(0, 10)
ax1.plot(np.sin(x))
ax2.plot(np.cos(x));
```



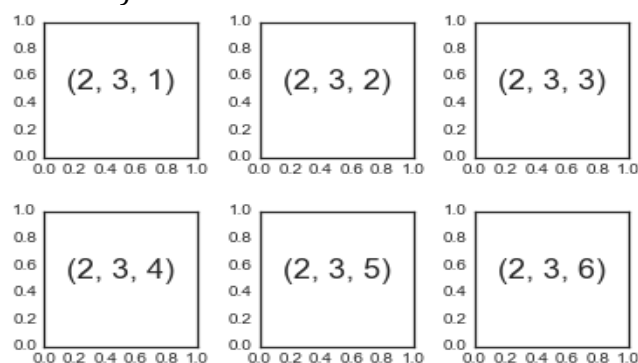
9.1 plt.subplot: Simple Grids of Subplots

Aligned columns or rows of subplots are a common enough need that Matplotlib has several convenience routines that make them easy to create. The lowest level of these is `plt.subplot`, which creates a single subplot within a grid. As you can see, this command takes three integer arguments—the number of rows, the number of columns, and the index of the plot to be created in this scheme, which runs from the upper left to the bottom right

```
In[4]: for i in range(1, 7):
        plt.subplot(2, 3, i)
        plt.text(0.5, 0.5, str((2, 3, i)), fontsize=18, ha='center') # Text coordinate
```



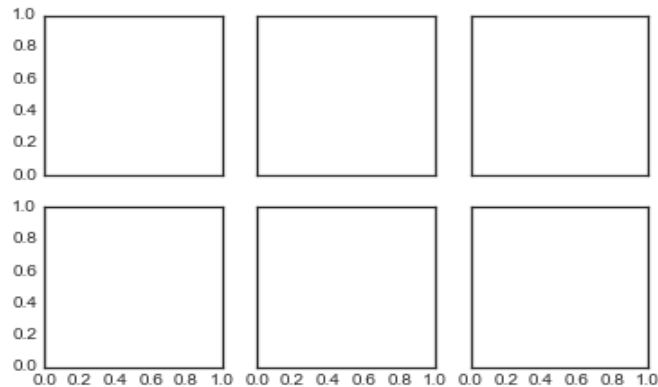
```
In[5]: fig = plt.figure()
        fig.subplots_adjust(hspace=0.4, wspace=0.4) # set spacing bw plots, height width
        spacing bw subplots
        for i in range(1, 7):
            ax = fig.add_subplot(2, 3, i)
            ax.text(0.5, 0.5, str((2, 3, i)),
                    fontsize=18, ha='center')
```



9.2 plt.subplots: The Whole Grid in One Go

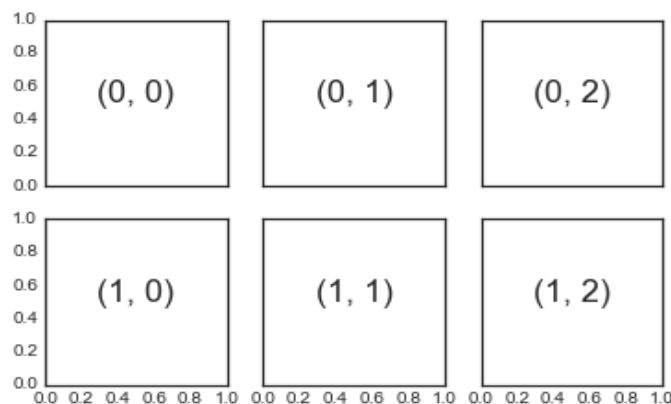
The approach just described quickly becomes tedious when creating a large grid of subplots, especially if you'd like to hide the x- and y-axis labels on the inner plots. For this purpose, `plt.subplots` is the easier tool to use (note the `s` at the end of `subplots`). Rather than creating a single subplot, this function creates a full grid of subplots in a single line, returning them in a NumPy array. The arguments are the number of rows and number of columns, along with optional keywords `sharex` and `sharey`, which allow you to specify the relationships between different axes.

In[6]: `fig, ax = plt.subplots(2, 3, sharex='col', sharey='row')`



In[7]: `# axes are in a two-dimensional array, indexed by [row, col]`

```
for i in range(2):
    for j in range(3):
        ax[i, j].text(0.5, 0.5, str((i, j)),
                      fontsize=18, ha='center')
fig
```

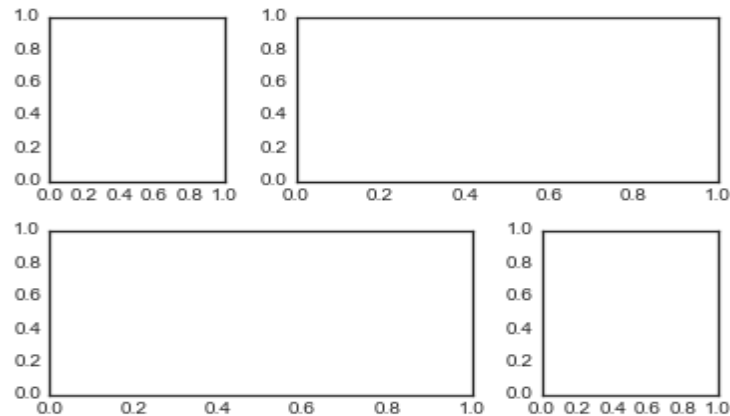


9.3 plt.GridSpec: More Complicated Arrangements

To go beyond a regular grid to subplots that span multiple rows and columns, `plt.GridSpec` is the best tool. `plt.GridSpec` does not create a plot by itself; it is rather a convenient interface that is recognized by the `plt.subplot` command. For example, a `GridSpec` for a grid of two rows and three columns with some specified width and height space looks like this

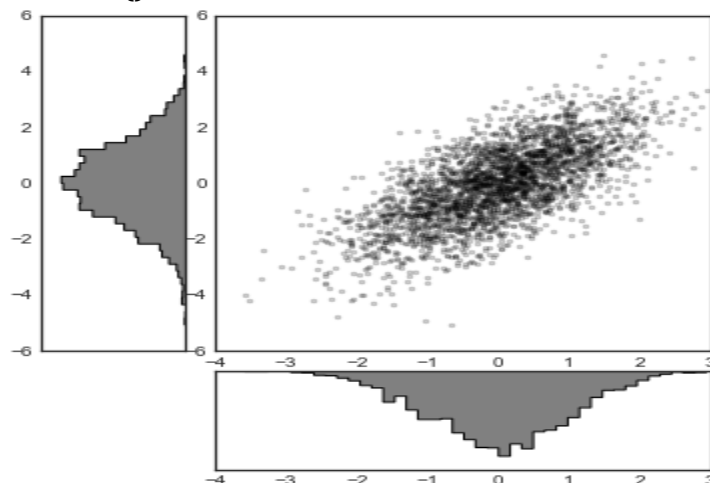
In[8]: `grid = plt.GridSpec(2, 3, wspace=0.4, hspace=0.3)`

```
In[9]: plt.subplot(grid[0, 0])
        plt.subplot(grid[0, 1:])
        plt.subplot(grid[1, :2])
        plt.subplot(grid[1, 2]);
```



```

In[10]: # Create some normally distributed data
mean = [0, 0]
cov = [[1, 1], [1, 2]]
x, y = np.random.multivariate_normal(mean, cov, 3000).T #3000 columns
# Set up the axes with gridspec
fig = plt.figure(figsize=(6, 6)) # width , height
grid = plt.GridSpec(4, 4, hspace=0.2, wspace=0.2) #row, column, grid layout to place
subplots within a figure
main_ax = fig.add_subplot(grid[:-1, 1:]) # from -1 till end of array
y_hist = fig.add_subplot(grid[:-1, 0], xticklabels=[], sharey=main_ax) #left fig
x_hist = fig.add_subplot(grid[-1, 1:], yticklabels=[], sharex=main_ax) # right fig
# scatter points on the main axes
main_ax.plot(x, y, 'ok', markersize=3, alpha=0.2) # center figure
# histogram on the attached axes
x_hist.hist(x, 40, histtype='stepfilled', orientation='vertical', color='gray')
x_hist.invert_yaxis()
y_hist.hist(y, 40, histtype='stepfilled', orientation='horizontal', color='gray')
y_hist.invert_xaxis()
    
```



10.TEXT AND ANNOTATION

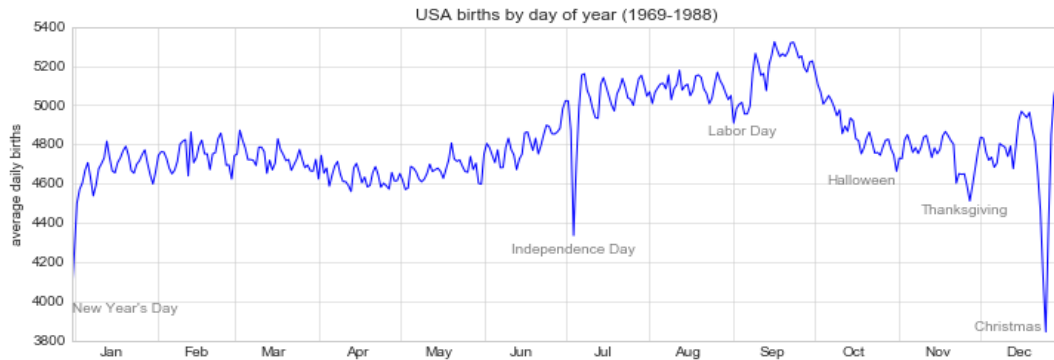
Creating a good visualization involves guiding the reader so that the figure tells a story. In some cases, this story can be told in an entirely visual manner, without the need for added text, but in others, small textual cues and labels are necessary. Perhaps the most basic types of annotations you will use are axes labels and titles, but the options go beyond this. Let's take a look at some data and how we might visualize and annotate it to help convey interesting information.

```
In[1]: %matplotlib inline
import matplotlib.pyplot as plt
import matplotlib as mpl
plt.style.use('seaborn-whitegrid')
import numpy as np
import pandas as pd
```

```
In[2]:
births = pd.read_csv('births.csv')
quartiles = np.percentile(births['births'], [25, 50, 75])
mu, sig = quartiles[1], 0.74 * (quartiles[2] - quartiles[0]) # mu, sig are variables
births = births.query('(births > @mu - 5 * @sig) & (births < @mu + 5 * @sig)')
#5 standard deviations from the mean
births['day'] = births['day'].astype(int)
births.index = pd.to_datetime(10000 * births.year + 100 * births.month + births.day,
format='%Y%m%d') # convert to date like object to date time objects
births_by_date = births.pivot_table('births', [births.index.month, births.index.day])
births_by_date.index = [pd.datetime(2012, month, day) for (month, day) in births_by_date.index]
In[3]: fig, ax = plt.subplots(figsize=(12, 4))
births_by_date.plot(ax=ax);
```



```
In[4]: fig, ax = plt.subplots(figsize=(12, 4))
births_by_date.plot(ax=ax)
# Add labels to the plot
style = dict(size=10, color='gray') # set label color size
ax.text('2012-1-1', 3950, "New Year's Day", **style) #kwargs
ax.text('2012-7-4', 4250, "Independence Day", ha='center', **style)
ax.text('2012-9-4', 4850, "Labor Day", ha='center', **style)
ax.text('2012-10-31', 4600, "Halloween", ha='right', **style)
ax.text('2012-11-25', 4450, "Thanksgiving", ha='center', **style)
ax.text('2012-12-25', 3850, "Christmas ", ha='right', **style)
# Label the axes
ax.set(title='USA births by day of year (1969-1988)', ylabel='average daily births')
# Format the x axis with centered month labels
ax.xaxis.set_major_locator(mpl.dates.MonthLocator()) #locate major ticks at the
beginning of each month
ax.xaxis.set_minor_locator(mpl.dates.MonthLocator(bymonthday=15)) # x axis tick is
placed on 15th of each month.
ax.xaxis.set_major_formatter(plt.NullFormatter()) # set major tick labels of the x-axis to
null (marked by red arrow)
ax.xaxis.set_minor_formatter(mpl.dates.DateFormatter('%h')); # month name is
displayed
```



Transforms and Text Position

- `ax.transData`

Transform associated with data coordinates

- `ax.transAxes`

Transform associated with the axes (in units of axes dimensions)

- `fig.transFigure`

Transform associated with the figure (in units of figure dimensions)

```
In[5]: fig, ax = plt.subplots(facecolor='lightgray')
```

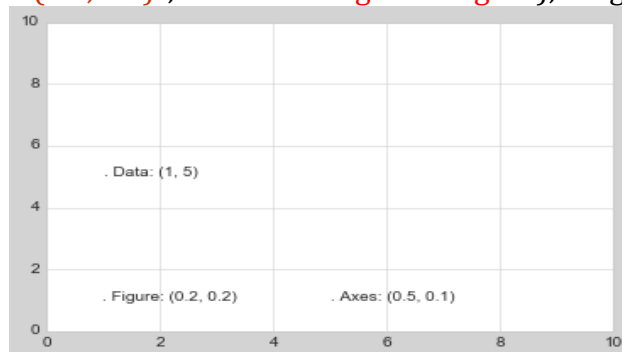
```
ax.axis([0, 10, 0, 10])
```

transform=ax.transData is the default, but we'll specify it anyway

```
ax.text(1, 5, ". Data: (1, 5)", transform=ax.transData)
```

```
ax.text(0.5, 0.1, ". Axes: (0.5, 0.1)", transform=ax.transAxes) # look at axis 5 and 1
```

```
ax.text(0.2, 0.2, ". Figure: (0.2, 0.2)", transform=fig.transFigure); # figure length, width
```



Arrows and Annotation

```
In[7]: %matplotlib inline
```

```
fig, ax = plt.subplots()
```

```
x = np.linspace(0, 20, 1000)
```

```
ax.plot(x, np.cos(x))
```

```
ax.axis('equal')
```

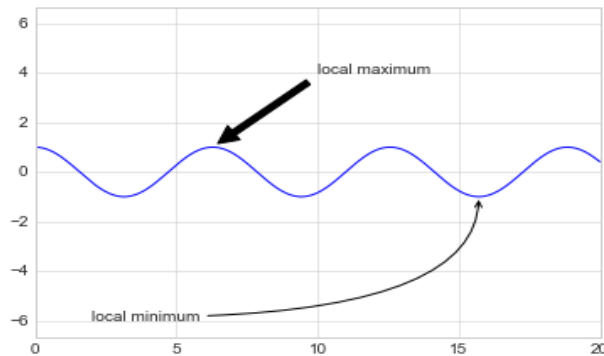
```
ax.annotate('local maximum', xy=(6.28, 1), xytext=(10, 4),
```

```
arrowprops=dict(facecolor='black', shrink=0.05)) #Text printed at position 10,4
```

```
ax.annotate('local minimum', xy=(5 * np.pi, -1), xytext=(2, -6),
```

```
arrowprops=dict(arrowstyle="->",
```

```
connectionstyle="angle3,angleA=0,angleB=-90"));
```



```

In[8]:
fig, ax = plt.subplots(figsize=(12, 4))
births_by_date.plot(ax=ax)
# Add labels to the plot
ax.annotate("New Year's Day", xy=('2012-1-1', 4100), xycoords='data',
xytext=(50, -30), textcoords='offset points',      #xytext position of text
arrowprops=dict(arrowstyle="->", connectionstyle="arc3,rad=-0.2"))
  
```



11. Customizing Ticks

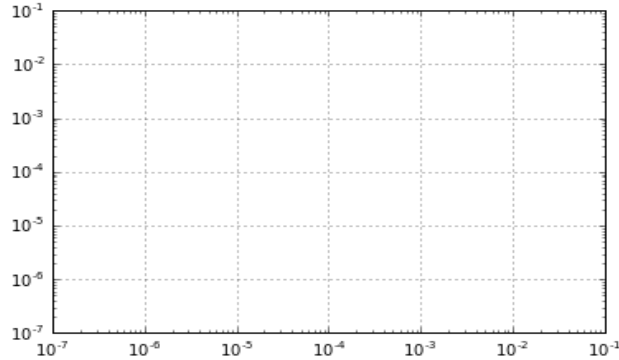
Matplotlib's default tick locators and formatters are designed to be generally sufficient in many common situations, but are in no way optimal for every plot. This chapter will give several examples of adjusting the tick locations and formatting for the particular plot type you're interested in. Before we go into examples, however, let's talk a bit more about the object hierarchy of Matplotlib plots. Matplotlib aims to have a Python object representing everything that appears on the plot: for example, recall that the Figure is the bounding box within which plot elements appear. Each Matplotlib object can also act as a container of subobjects: for example, each Figure can contain one or more Axes objects, each of which in turn contains other objects representing plot contents. The tickmarks are no exception. Each axes has attributes `xaxis` and `yaxis`, which in turn have attributes that contain all the properties of the lines, ticks, and labels that make up the axes.

11.1 Major and Minor Ticks

Within each axes, there is the concept of a *major* tickmark, and a *minor* tickmark. As the names imply, major ticks are usually bigger or more pronounced, while minor ticks are usually smaller. By default, Matplotlib rarely makes use of minor ticks, but one place you can see them is within logarithmic plots

```

In[1]: %matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('seaborn-whitegrid')
import numpy as np
In[2]: ax = plt.axes(xscale='log', yscale='log')
  
```

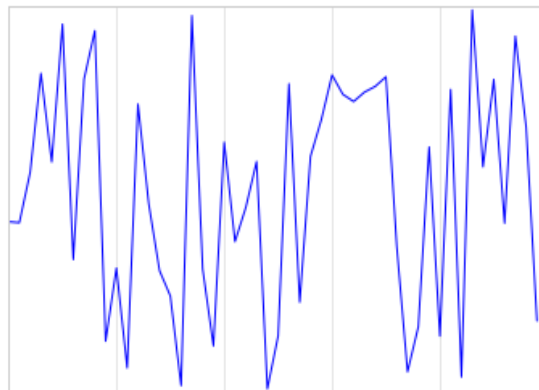


11.2. Hiding Ticks or Labels

Perhaps the most common tick/label formatting operation is the act of hiding ticks or labels. This can be done using `plt.NullLocator` and `plt.NullFormatter`, as shown here

```

In[5]: ax = plt.axes()
ax.plot(np.random.rand(50))
ax.yaxis.set_major_locator(plt.NullLocator()) #supresses y axis
ax.xaxis.set_major_formatter(plt.NullFormatter())
  
```



11.3 Reducing or Increasing the Number of Ticks

One common problem with the default settings is that smaller subplots can end up with crowded labels. We can see this in the plot grid shown here

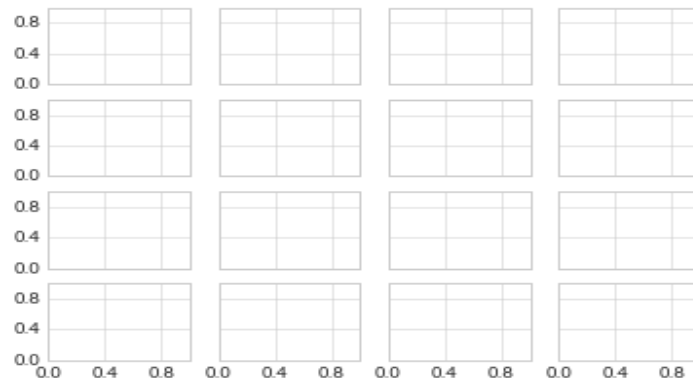
```

In[7]: fig, ax = plt.subplots(4, 4, sharex=True, sharey=True)
  
```



The above figure has crowded labels

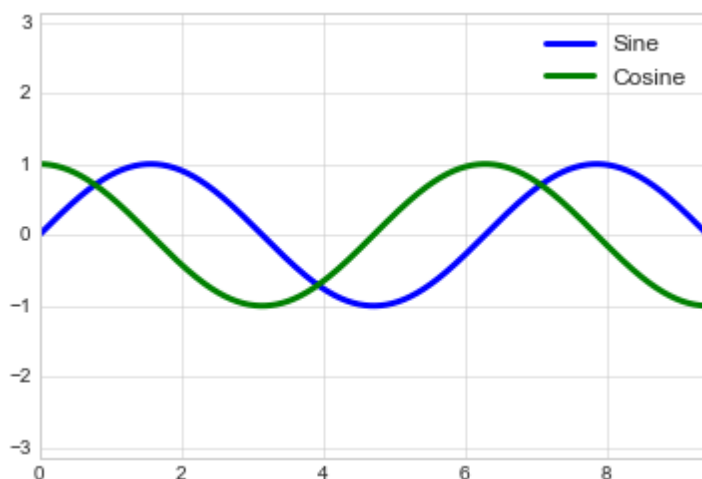
```
In[8]: for axi in ax.flat:
        axi.xaxis.set_major_locator(plt.MaxNLocator(3)) #set the number of ticks
        axi.yaxis.set_major_locator(plt.MaxNLocator(3))
        fig
```



11.4 Fancy Tick Formats

Matplotlib's default tick formatting can leave a lot to be desired: it works well as a broad default, but sometimes you'd like to do something different. Consider this plot of a sine and a cosine curve

```
In[9]: # Plot a sine and cosine curve
        fig, ax = plt.subplots()
        x = np.linspace(0, 3 * np.pi, 1000)
        ax.plot(x, np.sin(x), lw=3, label='Sine') #lw line width
        ax.plot(x, np.cos(x), lw=3, label='Cosine')
        # Set up grid, legend, and limits
        ax.grid(True)
        ax.legend(frameon=False)
        ax.axis('equal')
        ax.set_xlim(0, 3 * np.pi);
```



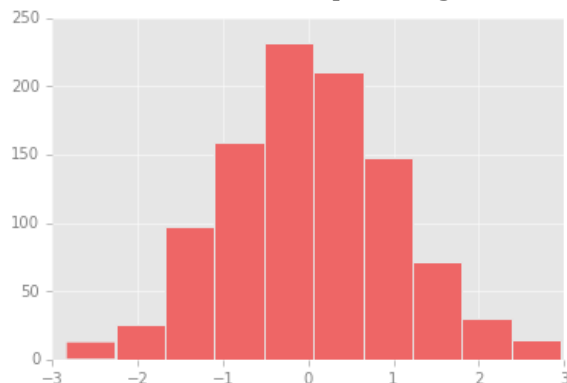
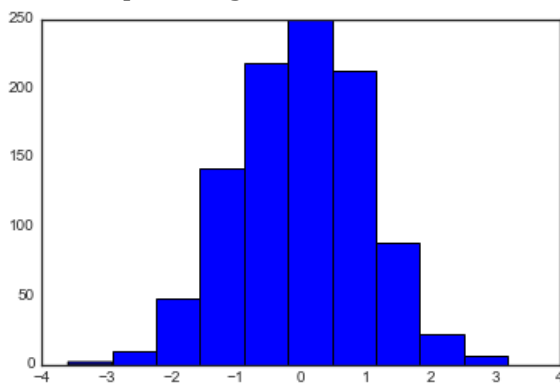
11.5 Customizing Matplotlib: Configurations and Stylesheets

Plot Customization by Hand First image shows a normal histogram to improve its quality
We do

```

In[3] :# use a gray background
ax = plt.axes(axisbg='#E6E6E6') # light shade of grey hexadecimal color code
ax.set_axisbelow(True)# Ticks and gridlines are below all Artists.
# draw solid white grid lines
plt.grid(color='w', linestyle='solid')
# hide axis spines
for spine in ax.spines.values():spine.set_visible(False) #disable border lines
# hide top and right ticks
ax.xaxis.tick_bottom()# hide ticks on top and bottom "-"
ax.yaxis.tick_left()
# lighten ticks and labels
ax.tick_params(colors='gray', direction='out')# tick colour and direction
for tick in ax.get_xticklabels():
    tick.set_color('gray')
for tick in ax.get_yticklabels():
    tick.set_color('gray')
# control face and edge color of histogram
ax.hist(x, edgecolor='#E6E6E6', color='#EE6666'); #bar edge colour and bar colour
  
```

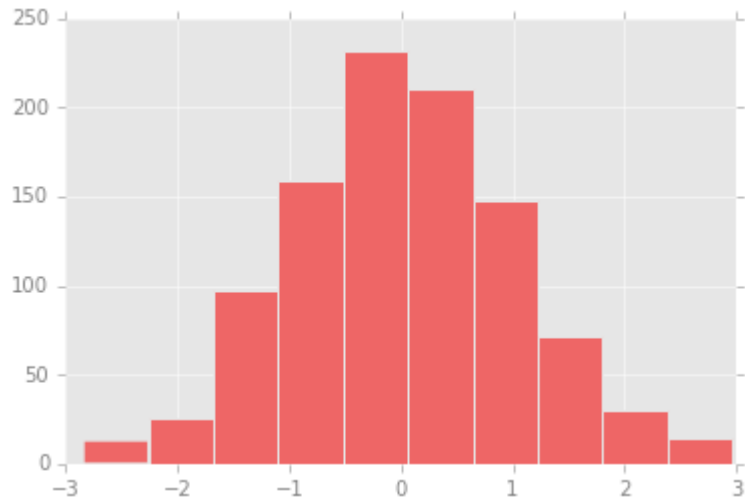
Input image



Since, this is hard to do all the modifications each time its best to change the defaults
Changing the Defaults: rcParams. Each time matplotlib loads it defines a runtime configuration (rc) containing default style for each plot. plt.rc.

```

In[4]: lPython_default = plt.rcParams.copy()
In[5]: from matplotlib importycler
        colors =ycler('color',
        ['#EE6666', '#3388BB', '#9988DD',
        '#EECC55', '#88BB44', '#FFBBBB'])
        plt.rc('axes', facecolor='#E6E6E6', edgecolor='none',
        axisbelow=True, grid=True, prop_cycle=colors)
        plt.rc('grid', color='w', linestyle='solid')
        plt.rc('xtick', direction='out', color='gray')
        plt.rc('ytick', direction='out', color='gray')
        plt.rc('patch', edgecolor='#E6E6E6')
        plt.rc('lines', linewidth=2)
In[6]: plt.hist(x);
  
```



```
In[7]: for i in range(4):
        plt.plot(np.random.rand(10)) # generate 10 random numbers
```



Stylesheets

```
In[8]: plt.style.available[:5] # names of the first five available Matplotlib styles
Out[8]: ['fivethirtyeight',
'seaborn-pastel',
'seaborn-whitegrid',
'ggplot',
'grayscale']
```

The basic way to switch to a stylesheet is to call:

```
plt.style.use('stylename')
with plt.style.context('stylename'):
    make_a_plot()
```

Let's create a function that will make two basic types of plot:

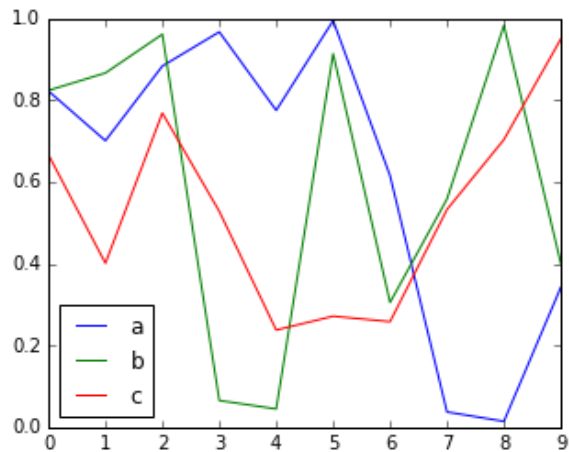
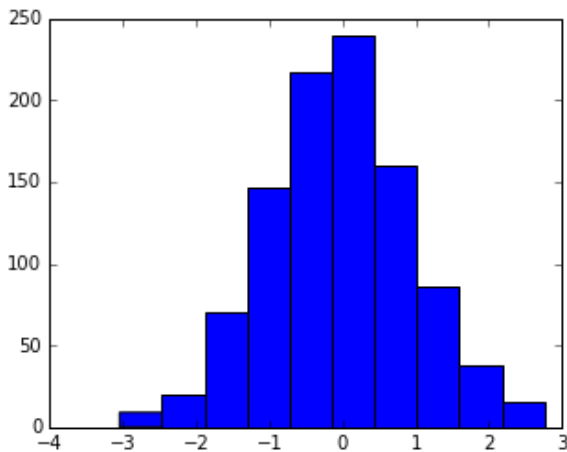
```
In[9]: def hist_and_lines():
        np.random.seed(0)
        fig, ax = plt.subplots(1, 2, figsize=(11, 4))
        ax[0].hist(np.random.randn(1000)) # plots histogram
        for i in range(3):
            ax[1].plot(np.random.rand(10))
        ax[1].legend(['a', 'b', 'c'], loc='lower left') # plots line graph
```

Default style

```
In[10]:# reset rcParams
plt.rcParams.update(IPython_default);
```

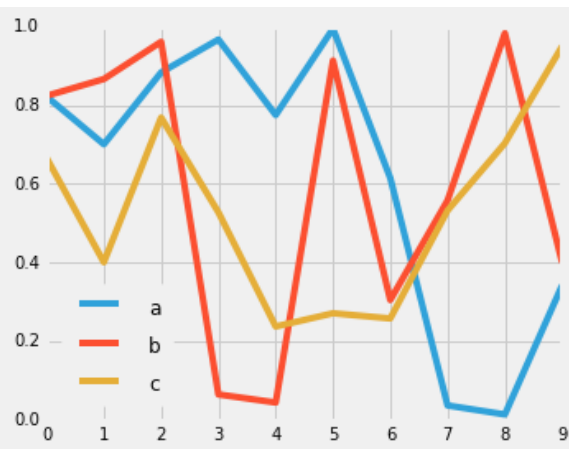
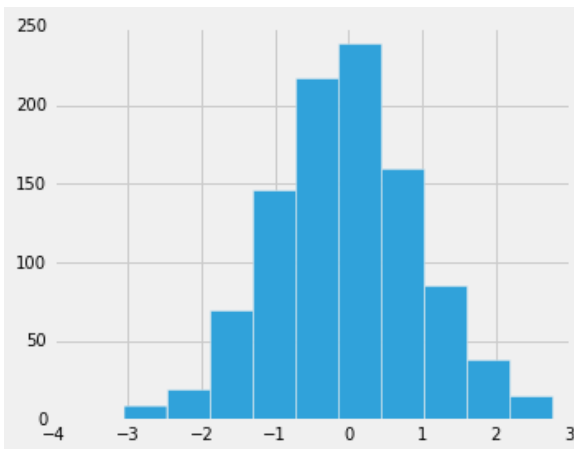
Now let's see how it looks (Figure 4-85):

```
In[11]:hist_and_lines()
```



FiveThirtyEight style

```
In[12]:with plt.style.context('fivethirtyeight'):
hist_and_lines()
```



Similarly, we have ggplot, Bayesian Methods for Hackers style, Dark background, Grayscale, Seaborn style

12.Three-Dimensional Plotting in Matplotlib

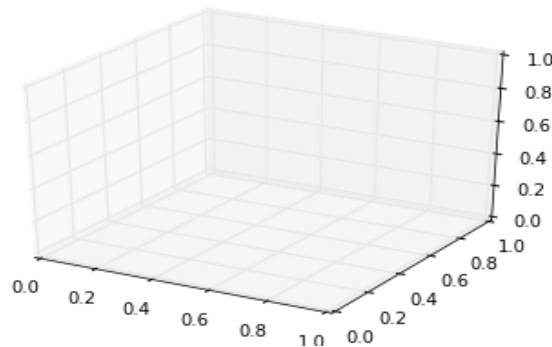
Matplotlib was initially designed with only two-dimensional plotting in mind. Around the time of the 1.0 release, some three-dimensional plotting utilities were built on top of Matplotlib's two-dimensional display, and the result is a convenient (if somewhat limited) set of tools for three-dimensional data visualization. Three-dimensional plots are enabled by importing the mplot3d toolkit, included with the main Matplotlib installation

```
In[1]: from mpl_toolkits import mplot3d
```

```

In[2]: %matplotlib inline
import numpy as np
import matplotlib.pyplot as plt

In[3]: fig = plt.figure()
ax = plt.axes(projection='3d')
  
```

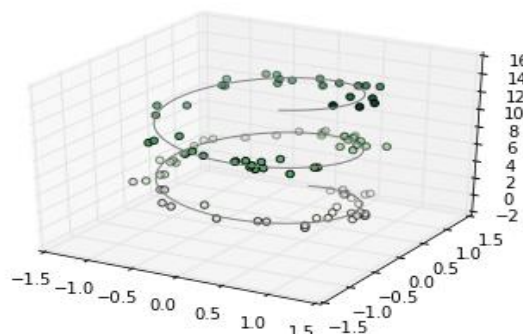


12.1 Three-Dimensional Points and Lines

The most basic three-dimensional plot is a line or collection of scatter plots created from sets of (x, y, z) triples. In analogy with the more common two-dimensional plots discussed earlier, these can be created using the `ax.plot3D` and `ax.scatter3D` functions. The call signature for these is nearly identical to that of their two-dimensional counterparts, so you can refer to Simple Line Plots and Simple Scatter Plots for more information on controlling the output. Here we'll plot a trigonometric spiral, along with some points drawn randomly near the line

```

In[4]: ax = plt.axes(projection='3d')
        # Data for a three-dimensional line
        zline = np.linspace(0, 15, 1000)
        xline = np.sin(zline)
        yline = np.cos(zline)
        ax.plot3D(xline, yline, zline, 'gray')
        # Data for three-dimensional scattered points
        zdata = 15 * np.random.random(100)
        xdata = np.sin(zdata) + 0.1 * np.random.randn(100)
        ydata = np.cos(zdata) + 0.1 * np.random.randn(100)
        ax.scatter3D(xdata, ydata, zdata, c=zdata, cmap='Greens'); #scatter points
  
```



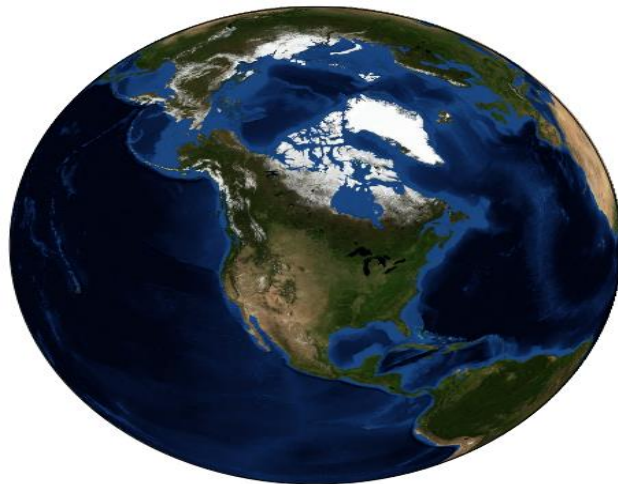
13. Geographic Data with Basemap

Analogous to the contour plots we explored in Density and Contour Plots, mplot3d contains tools to create three-dimensional relief plots using the same inputs. Like `ax.contour`, `ax.contour3D` requires all the input data to be in the form of two-dimensional regular grids, with the `z` data evaluated at each point. Here we'll show a three-dimensional contour diagram of a three-dimensional sinusoidal function

\$ conda install basemap

```
In[1]: %matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.basemap import Basemap
```

```
In[2]: plt.figure(figsize=(8, 8))
m = Basemap(projection='ortho', resolution=None, lat_0=50, lon_0=-100)
m.bluemarble(scale=0.5);
```



```
In[3]: fig = plt.figure(figsize=(8, 8))
m = Basemap(projection='lcc', resolution=None, width=8E6, height=8E6, lat_0=45,
lon_0=-100,)
m.etopo(scale=0.5, alpha=0.5)      #satellite image
# Map (long, lat) to (x, y) for plotting
x, y = m(-122.3, 47.6)             #lat and long
plt.plot(x, y, 'ok', markersize=5)
plt.text(x, y, 'Seattle', fontsize=12);
```



Drawing a Map Background

- Physical boundaries and bodies of water

`drawcoastlines()`

Draw continental coast lines

`drawlsmask()`

Draw a mask between the land and sea, for use with projecting images on one or the other

`drawmapboundary()`

Draw the map boundary, including the fill color for oceans

`drawrivers()`

Draw rivers on the map

`fillcontinents()`

Fill the continents with a given color; optionally fill lakes with another color

- Political boundaries

`drawcountries()`

Draw country boundaries

`drawstates()`

Draw US state boundaries

`drawcounties()`

Draw US county boundaries

- Map features

`drawgreatcircle()`

Draw a great circle between two points

`drawparallels()`

Draw lines of constant latitude

`drawmeridians()`

Draw lines of constant longitude

`drawmapscale()`

Draw a linear scale on the map

- Whole-globe images

`bluemarble()`

Project NASA's blue marble image onto the map

`shadedrelief()`

Project a shaded relief image onto the map
 etopo()
 Draw an etopo relief image onto the map
 warpimage()
 Project a user-provided image onto the map

Plotting Data on Maps

contour()/contourf()
 Draw contour lines or filled contours
 imshow()
 Draw an image
 pcolor()/pcolormesh()
 Draw a pseudocolor plot for irregular/regular meshes
 plot()
 Draw lines and/or markers
 scatter()
 Draw points with markers
 quiver()
 Draw vectors
 barbs()
 Draw wind barbs
 drawgreatcircle()
 Draw a great circle

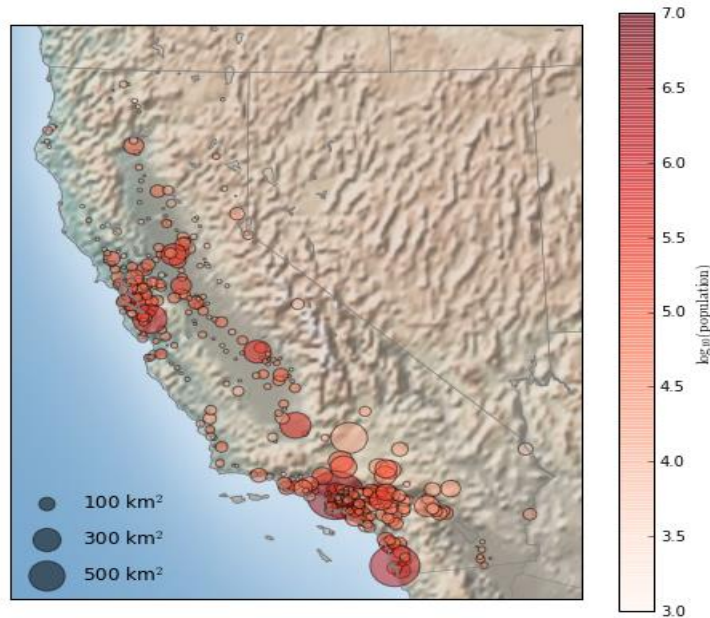
Example: California Cities

```
In[10]: import pandas as pd
        cities = pd.read_csv('data/california_cities.csv')
        # Extract the data we're interested in
        lat = cities['latd'].values
        lon = cities['longd'].values
        population = cities['population_total'].values
        area = cities['area_total_km2'].values

In[11]: # 1. Draw the map background
        fig = plt.figure(figsize=(8, 8))
        m = Basemap(projection='lcc', resolution='h', # map projection, resolution high
        lat_0=37.5, lon_0=-119,
        width=1E6, height=1.2E6)
        m.shadedrelief() #draw shaded satellite image
        m.drawcoastlines(color='gray')
        m.drawcountries(color='gray')
        m.drawstates(color='gray')
        # 2. scatter city data, with color reflecting population
        # and size reflecting area
        m.scatter(lon, lat, latlon=True, c=np.log10(population), s=area, cmap='Reds',
        alpha=0.5)
        # 3. create colorbar and legend
        plt.colorbar(label=r'$\log_{10}(\text{population})$')
```

```

plt.clim(3, 7) # Set the color limits of the current image.
# make legend with dummy points
for a in [100, 300, 500]:
    plt.scatter([], [], c='k', alpha=0.5, s=a,
        label=str(a) + ' km$^2$')
plt.legend(scatterpoints=1, frameon=False,
    labelspacing=1, loc='lower left');
  
```



14. Visualization with Seaborn

Matplotlib has been at the core of scientific visualization in Python for decades, but even avid users will admit it often leaves much to be desired. There are several complaints about Matplotlib that often come up:

- A common early complaint, which is now outdated: prior to version 2.0, Matplotlib's color and style defaults were at times poor and looked dated.
- Matplotlib's API is relatively low-level. Doing sophisticated statistical visualization is possible, but often requires a *lot* of boilerplate code.
- Matplotlib predated Pandas by more than a decade, and thus is not designed for use with Pandas DataFrame objects. In order to visualize data from a DataFrame, you must extract each Series and often concatenate them together into the right format. It would be nicer to have a plotting library that can intelligently use the DataFrame labels in a plot.

An answer to these problems is Seaborn. Seaborn provides an API on top of Matplotlib that offers sane choices for plot style and color defaults, defines simple high-level functions for common statistical plot types, and integrates with the functionality provided by Pandas.

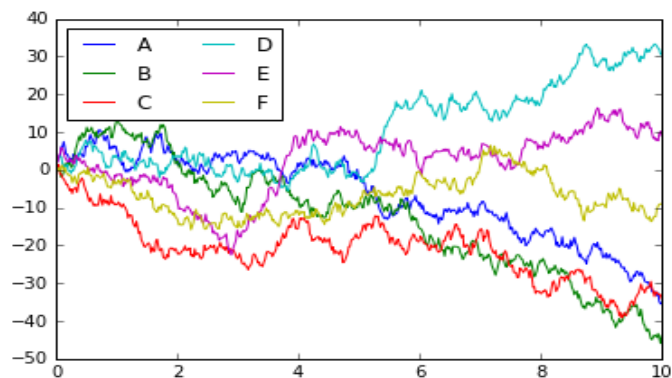
To be fair, the Matplotlib team has adapted to the changing landscape: it added the `plt.style` tools discussed in Customizing Matplotlib: Configurations and Style Sheets, and Matplotlib is starting to handle Pandas data more seamlessly. But for all the reasons just discussed, Seaborn remains a useful add-on.

Example of matplotlib lib classic plot.

```
In[1]: import matplotlib.pyplot as plt
plt.style.use('classic')
%matplotlib inline
import numpy as np
import pandas as pd
```

```
In[2]: # Create some data
rng = np.random.RandomState(0)
x = np.linspace(0, 10, 500)
y = np.cumsum(rng.randn(500, 6), 0)    #cumulative sum of elements (partial sum
of sequence)
```

```
In[3]: # Plot the data with Matplotlib defaults
plt.plot(x, y)
plt.legend('ABCDEF', ncol=2, loc='upper left');
```



Seaborn image plot

```
In[4]: import seaborn as sns
sns.set() # Seaborn's default settings to your plots,
```

```
In[5]: # same plotting code as above!
plt.plot(x, y)
plt.legend('ABCDEF', ncol=2, loc='upper left');
```

